VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING AND ITS APPLICATIONS

CO3133 – SEMESTER-261

ASSIGNMENT 1 – FINAL REPORT

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Class: CC01

Instructor: Dr. Le Thanh Sach

Group: Tiki-taka

No.	Full Name	Student ID
1	Nguyen Anh Khoa	2452539
2	Nguyen Hao Thien	2453194
3	Ta Tuan Khai	2452515

Ho Chi Minh City, October, 2026

WORK DISTRIBUTION

Course: Deep Learning and Its Applications – CO3133
Assignment: Assignment 1 – Final Report
Class: CC01
Instructor: Dr. Le Thanh Sach
Group: Tiki-taka

No.	Name	Student ID	Assigned Tasks	Completion
1	Nguyen Anh Khoa	2452539	Shared engine, MLP/GRU, metrics, checkpoints, and timing	100%
2	Nguyen Hao Thien	2453194	Dataset, EDA, preprocessing, Linear/CNN, and error analysis	100%
3	Ta Tuan Khai	2452515	Transformer, configurations, artifacts, integration, and report	100%



Abstract

Across 15 Fashion-MNIST training runs, the custom Transformer achieves the highest mean test accuracy and macro-F1, 91.267% and 91.234%, narrowly ahead of the custom CNN at 90.690% and 90.726%.

The comparison uses five architecture families, one persisted stratified split and three seeds per configuration. Selected checkpoints are evaluated on the official 10,000-image test source, with mean and sample standard deviation reported across seeds.

The CNN offers close predictive performance with about 72.9% fewer parameters than the Transformer and lower observed training-loop cost. Separately measured Windows CPU inference favors different trade-offs; the model choice depends on quality, size and the intended runtime.

All classifiers use shared preprocessing and metrics, while their representations range from flattened pixels to spatial feature maps, image rows and patch tokens. The study reports validation/test metrics, exact parameter counts, learning curves, T4 train-and-validation loop times, explicitly labeled CPU timing, confusion matrices and selected successes/-failures. Shirt and related upper-body categories remain difficult across models. Small seed count, different training policies and separate timing environments prevent a causal or universal ranking of model families. Effective configurations, run/checkpoint identities, verification records and reconstruction instructions accompany the editable report source.



Table of Contents

Abstract **i**

1 Problem and Data Description **1**
1.1 Problem Formulation and Research Questions 1
1.2 Dataset Description and Provenance 1
1.3 Exploratory Data Analysis 2
1.4 Data Splitting and Leakage Prevention 4
1.5 Preprocessing, Augmentation and Example Batch 4

2 Methodology **6**
2.1 End-to-End Pipeline and Data Loading 6
2.2 Shared Model Interface and Input Representations 6
2.3 Model Architectures 7
2.3.1 Linear / Softmax Classifier 7
2.3.2 Multilayer Perceptron 7
2.3.3 Custom Convolutional Neural Network 8
2.3.4 GRU-Based Image Classifier 9
2.3.5 Custom Transformer Classifier 9
2.4 Loss Function and Evaluation Metrics 10
2.5 Training, Hyperparameter Selection and Checkpointing 11
2.6 Experimental Design and Fairness Controls 11
2.7 Environment, Timing and Reproduction Protocol 11

3 Implementation Results **13**
3.1 Completed Runs and Evidence Coverage 13
3.2 Main Quantitative Comparison 13
3.3 Learning Dynamics and Convergence 15
3.4 Efficiency and Accuracy–Compute Trade-offs 18
3.5 Confusion Matrices and Class-Level Results 19
3.5.1 Linear 20
3.5.2 MLP 21
3.5.3 CNN 22
3.5.4 GRU 23



3.5.5 Transformer 24
3.6 Qualitative Predictions and Error Analysis 25
3.7 Representation and Inductive-Bias Discussion 30
3.8 Limitations 30
3.9 Conclusion 31

Reproducibility and Artifact Access **32**

Member Contributions and AI Usage Disclosure **33**

A Full Configurations and Architecture Details **36**

B Per-Seed Results and Run Provenance **36**

C Additional Class-Level Results **38**

D Detailed Verification and Reproduction Notes **40**

1 Problem and Data Description

1.1 Problem Formulation and Research Questions

Low-resolution product images provide a controlled setting for studying how model structure affects classification. Fashion-MNIST includes visually similar garments, so success requires distinguishing shape and texture rather than exploiting a class frequency. We compare five implementations on a fixed partition and evaluation protocol.
Each input is a grayscale image $X_i \in \mathbb{R}^{1\times 28\times 28}$ with one label $y_i \in \{0,\dots,9\}$. A classifier produces ten raw scores:

$$f_\theta(X_i) = z_i \in \mathbb{R}^{10}, \qquad \widehat{y}_i = \arg\max_{k\in\{0,\dots,9\}} z_{i,k}.$$

The objective is mean multiclass cross-entropy; probabilities are recorded for interpretation, not supplied to the loss. The prediction unit is one image. The comparison addresses four questions:

1. How do Linear, MLP, CNN, GRU and Transformer configurations compare in held-out test accuracy and macro-F1?
2. How do flattened, spatial, row-sequence and patch-sequence representations relate to their strengths and class-specific errors?
3. What trade-offs arise among predictive quality, parameter count, training cost and measured CPU inference cost?
4. How much do results vary across training seeds 69420, 67 and 69?

The results describe the tested configurations. Parameter count, optimization policy and regularization also vary, so differences cannot be attributed solely to architecture.

1.2 Dataset Description and Provenance

Fashion-MNIST is the Zalando Research benchmark introduced by Xiao et al.[1] Its official sources contain 60,000 training and 10,000 test images, each an 8-bit 28×28 grayscale array, with ten labels and 7,000 examples per class across both sources. The official repository publishes the dataset under the MIT license.[2] Our loaders use `torchvision.datasets.FashionMNIST` and the original IDX resources, without changing the label mapping. CSV exports are not used by the training pipeline.
The original authors derive the benchmark from front-view catalog thumbnails. Their conversion includes background-border trimming, subsampling, sharpening, canvas centering,

intensity inversion and grayscale conversion.[1] These are dataset-construction operations. The group starts from the distributed IDX images and applies the separate transformations in Section 1.5. This distinguishes the dataset authors' processing from our code.

Raw IDX images have a 16-byte header and label files an 8-byte header. Table 1.1 records byte counts of the cached canonical resources used for report verification. Each gzip resource was checked against the MD5 value in torchvision's resource list, and decompression matched the IDX file byte-for-byte. These are actual gzip sizes, rather than a generic archive-viewer size column. The four uncompressed IDX files occupy 54,950,048 bytes.

Table 1.1: Verified canonical Fashion-MNIST resources in the local cache.

Canonical resource	IDX bytes	Gzip bytes
train-images images	47,040,016	26,421,880
train-labels labels	60,008	29,515
t10k-images images	7,840,016	4,422,102
t10k-labels labels	10,008	5,148
Total	54,950,048	30,878,645

This inventory describes the cached resources; it does not assert an undocumented Kaggle acquisition session.

Limitations include small grayscale inputs, catalog-style backgrounds and overlapping visual definitions among tops, shirts, pullovers and coats. Balanced label counts do not remove content bias, annotation ambiguity or the domain gap from real photographs.

1.3 Exploratory Data Analysis

The saved stratified partition has exactly 5,000 training and 1,000 validation examples per class; each official-test class also has 1,000 examples. Table 1.2 was checked against the saved indices and raw labels. The largest-to-smallest count ratio is 1.0 in each partition, so label imbalance does not motivate resampling or class weighting here.

Figure 1.1 shows the first four saved training indices of each category. These are unaugmented training images, not examples chosen using test performance. Their variation and shared upper-body silhouettes suggest plausible sources of confusion. Visual similarity alone does not establish prediction confidence. Actual probabilities and confusion frequencies are examined in Sections 3.5 and 3.6. No manual relabeling, perceptual duplicate removal or custom outlier filtering was performed.

Table 1.2: Class support in the fixed seed-36 split.

ID	Class	Train	Validation	Test
0	T-shirt/top	5000	1000	1000
1	Trouser	5000	1000	1000
2	Pullover	5000	1000	1000
3	Dress	5000	1000	1000
4	Coat	5000	1000	1000
5	Sandal	5000	1000	1000
6	Shirt	5000	1000	1000
7	Sneaker	5000	1000	1000
8	Bag	5000	1000	1000
9	Ankle boot	5000	1000	1000

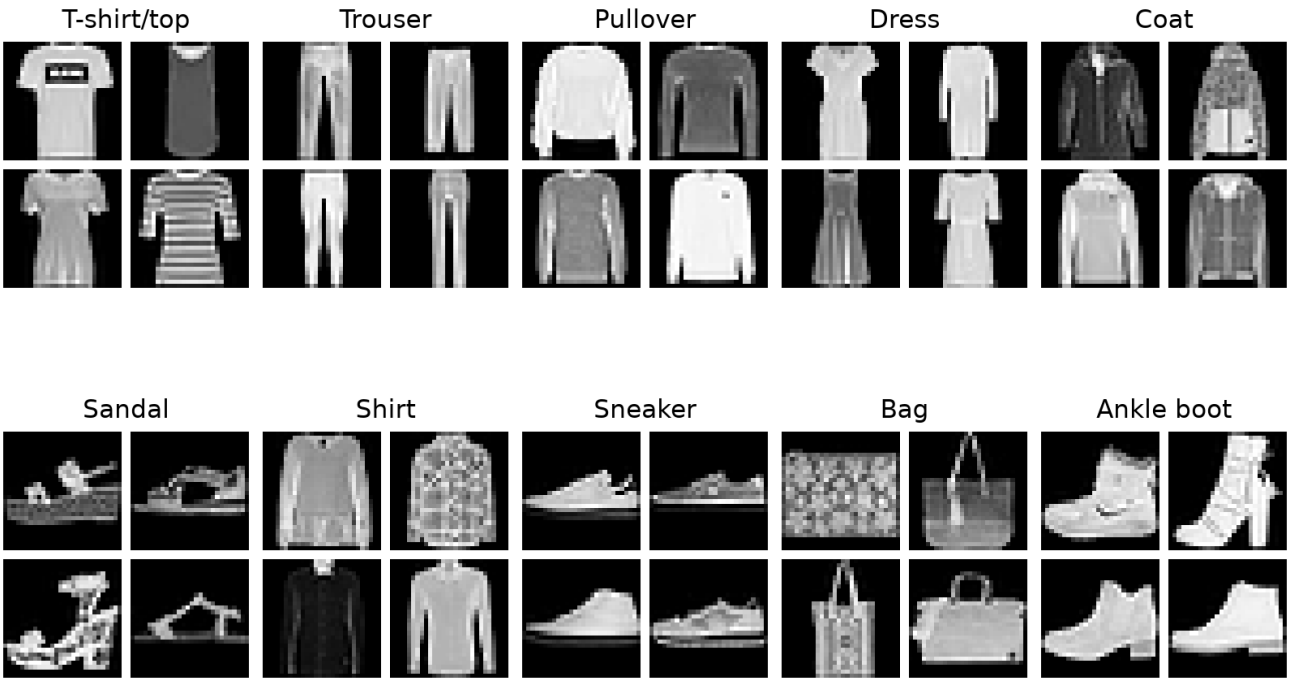


Figure 1.1: Forty representative training images: four examples per class panel. Selection IDs are retained in `evidence/eda_selection.json`.

1.4 Data Splitting and Leakage Prevention

The official training source is stratified into 50,000 training and 10,000 validation images with split seed 36. The official 10,000-example test source remains separate. The persisted manifest is `configs/a1/splits/fashion_mnist_split.json`; ordinary loader construction reads it instead of resampling a new split.

The validator checks index ranges, duplicates, full source coverage and disjoint training/-validation membership. Test indices use a different source namespace even when their integer values coincide with training indices. Normalization is fitted on unaugmented training indices only. Validation selects checkpoints; final test scores do not select a checkpoint or the best seed.

The short excerpt below uses the implementation's actual manifest fields:

```
train_indices = manifest.train_indices
val_indices = manifest.validation_indices
test_indices = manifest.test_indices
```

These controls prevent index overlap and normalization leakage. They do not establish that visually duplicated product images have been removed across partitions; no perceptual-duplicate audit is claimed.

1.5 Preprocessing, Augmentation and Example Batch

All five main configurations use the same training-only random crop: 28×28 output after two pixels of zero-valued constant padding per side. No horizontal flip is enabled. Tensor conversion scales uint8 pixels to $[0, 1]$, then applies the training-derived statistics:

$$\mu = 0.28585961086061534, \qquad \sigma = 0.352790375749525, \qquad x' = (x - \mu)/\sigma.$$

Validation and test apply tensor conversion and this normalization without random augmentation. Their values do not contribute to fitting the statistics.

Every reported main run has configured batch size 64 and retains the last incomplete batch. The shared contract is float32 images $[B, 1, 28, 28]$, int64 labels $[B]$, and B stable sample IDs. For instance, `fashion_mnist:official_test:0` identifies an original test image, not a batch-relative position.

Figure 1.2 shows six entries of a 64-example batch regenerated for the report from saved training indices, using crop RNG seed 69420 and the same transform definition. This is a post-hoc data illustration, not a logged training-epoch batch. Its actual

shape is $[64, 1, 28, 28]$, dtype float32, and observed normalized range approximately $[-0.8103, 2.0243]$. The display maps that fixed range to grayscale.

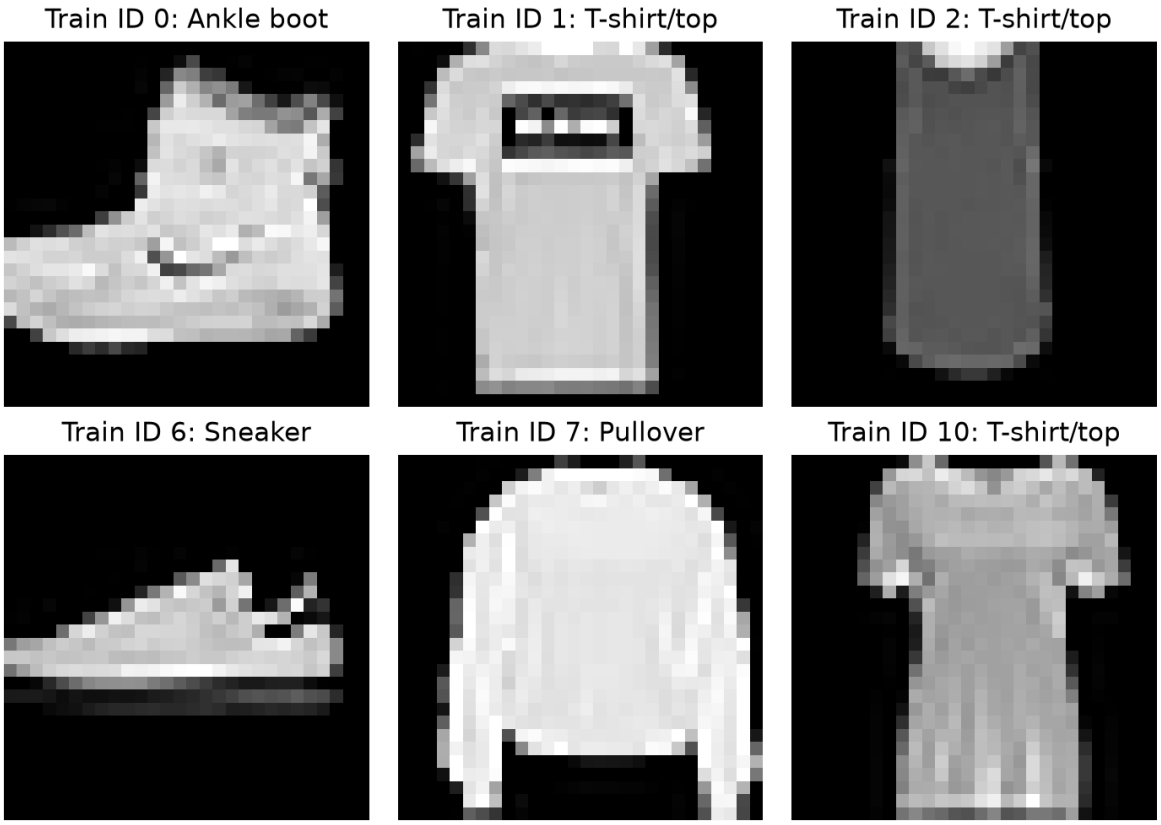


Figure 1.2: Post-hoc example training batch after crop and normalization. Six of 64 images are shown; IDs/statistics are saved in `evidence/example_batch.json`.

2 Methodology

2.1 End-to-End Pipeline and Data Loading

The pipeline uses a custom subset wrapper and collation function around torchvision's raw Fashion-MNIST dataset. Training and validation wrappers use disjoint official-training indices and independent transform objects; the test wrapper uses the official test source. Training shuffles through a dedicated run-seeded generator. Validation and test do not shuffle. All loaders retain IDs and use `drop_last=False`.

Figure 2.1 separates the update and evaluation paths. Training enables dropout and gradients; validation/test use evaluation mode and inference without gradient tracking. Both consume the same image/label batch contract. The engine computes sample-weighted loss

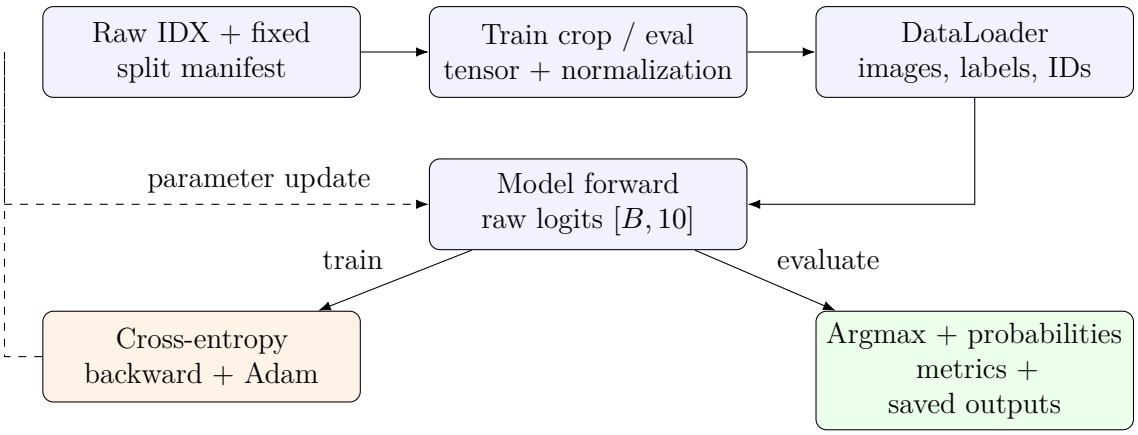


Figure 2.1: Data preparation and model interface, with distinct update/evaluation paths. Softmax probabilities support analysis; cross-entropy receives raw logits.

and epoch-level accuracy/F1 from online training predictions. These include augmentation and training-mode behavior. Thus, close train/validation curves alone do not prove absence of overfitting.

2.2 Shared Model Interface and Input Representations

Every classifier accepts float32 $[B, 1, 28, 28]$ and returns raw $[B, 10]$ logits. Representation changes occur inside the model, leaving data loading and metrics unchanged. Table 2.1 connects each representation to its structural preference. The input shape and label space are shared despite different learned representations and parameter counts.

Table 2.1: Representations used in the reported configurations.

Model	Internal representation	Structural preference
Linear	Flattened 784-vector	One affine decision rule per class
MLP	Flattened vector, dense hidden features	Nonlinear global combinations
CNN	Two-dimensional feature maps	Local connectivity and shared filters
GRU	28 row timesteps of width 28	Ordered recurrent context
Transformer	49 4×4 patches, width 64	Position-aware token interactions

2.3 Model Architectures

2.3.1 Linear / Softmax Classifier

The baseline flattens each image and applies one affine layer:

$$x = \text{flatten}(X) \in \mathbb{R}^{784}, \qquad z = Wx + b, \quad W \in \mathbb{R}^{10\times 784}.$$

There is no hidden nonlinearity. The $784 \times 10 + 10 = 7,850$ parameters are all trainable; flattening adds none. Probabilities, when needed, are $p_k = \exp(z_k)/\sum_{j=0}^{9}\exp(z_j)$. The implementation returns logits because cross-entropy already combines log-softmax and the label loss.[5] The project implements the wrapper and validation around PyTorch `nn.Flatten` and `nn.Linear`. This small baseline has neither a hidden feature hierarchy nor built-in image locality.

2.3.2 Multilayer Perceptron

The MLP uses $784 \to 256 \to 128 \to 10$. Every hidden affine layer is followed by ReLU and dropout 0.1; the final layer returns logits. For hidden layer ℓ,

$$a_\ell = W_\ell h_{\ell-1} + b_\ell, \qquad h_\ell = \text{ReLU}(a_\ell).$$

ReLU keeps positive values and sets negative values to zero, enabling nonlinear boundaries. Without nonlinearities, consecutive affine layers collapse into one affine transformation. Training-time dropout masks hidden activations and scales retained values; evaluation disables these masks. It regularizes dense features, but does not favor spatially adjacent pixels. The parameter count is

$$(784 \cdot 256 + 256) + (256 \cdot 128 + 128) + (128 \cdot 10 + 10) = 235,146.$$

Project code assembles the network from PyTorch layers. Other activation names are supported, but only ReLU is used in the reported MLP runs; no activation ablation is claimed.

2.3.3 Custom Convolutional Neural Network

Convolutions apply learned filters to local neighborhoods, reusing weights across positions. Multiple filters yield feature maps responsive to patterns learned from data. For stride one, a filter response has the form

$$Z_{m,i,j} = b_m + \sum_c \sum_u \sum_v W_{m,c,u,v} X_{c,i+u,j+v}.$$

For illustration, a 2×2 filter with ones on its main diagonal gives $1+6=7$ on $\left[\begin{smallmatrix} 1 & 2 \\ 5 & 6 \end{smallmatrix}\right]$. This is a numerical example; the actual network uses 3×3 kernels.

Max pooling retains the largest value in a region: the same example pools to 6, versus 3.5 under average pooling. Pooling reduces spatial resolution and may reduce sensitivity to small shifts, while discarding fine spatial detail. Its aggregation rule has no trainable weights.

Table 2.2 gives the actual custom architecture. Padding preserves each convolution's size; max pooling reduces spatial dimensions $28 \to 14 \to 7$. The implementation uses `ceil_mode=True`; these even-sized inputs have the dimensions shown. Adaptive average pooling gives one value per channel. All 28,426 parameters are trainable. The project

Table 2.2: Custom CNN layers and resulting tensor shapes.

Layer	Output
Input	$[B, 1, 28, 28]$
Conv $1 \to 32$, 3×3, pad 1; ReLU	$[B, 32, 28, 28]$
MaxPool 2×2	$[B, 32, 14, 14]$
Conv $32 \to 64$, 3×3, pad 1; ReLU	$[B, 64, 14, 14]$
MaxPool 2×2	$[B, 64, 7, 7]$
AdaptiveAvgPool; flatten	$[B, 64]$
Linear $64 \to 128$; ReLU; Dropout 0.1	$[B, 128]$
Linear $128 \to 10$	$[B, 10]$

defines this network using PyTorch convolution, activation, pooling and linear layers; it is not a pretrained CNN backbone.

2.3.4 GRU-Based Image Classifier

Each image row is one timestep,

$$x_t = X_{t,:} \in \mathbb{R}^{28}, \qquad t = 1, \ldots, 28.$$

The batch-first sequence has shape $[B, 28, 28]$. A single unidirectional `nn.GRU` uses input width 28 and hidden width 128; its final hidden state is projected to ten logits. A timestep is a complete row, not one pixel. The traversal carries information from earlier rows into later decisions.

The equations below match the PyTorch cell actually used:[4]

$$\begin{aligned}
r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),\\
z_t &= \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}),\\
n_t &= \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})),\\
h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1}.
\end{aligned}$$

The reset gate modulates transformed past information in the candidate; the update gate blends it with retained memory. PyTorch multiplies by the reset gate after the hidden affine transformation.

Recurrent weights and two bias vectors contribute $3(28 \cdot 128 + 128 \cdot 128 + 2 \cdot 128) = 60,672$ parameters. The head adds 1,290, totaling 61,962. The project wrapper handles row conversion and state selection, while the cell is a PyTorch component. Although the code supports columns, LSTM and bidirectionality, these variants were not evaluated. The reported single-layer GRU has no inter-layer dropout transition.

2.3.5 Custom Transformer Classifier

The custom Transformer divides an image into a 7×7 grid of non-overlapping 4×4 patches. The 49 patches are flattened to 16 values each and projected to embedding width 64. A learned 49×64 position embedding is added:

$$P \in \mathbb{R}^{B \times 49 \times 16}, \qquad H_0 = PW_E + b_E + E_{\text{pos}} \in \mathbb{R}^{B \times 49 \times 64}.$$

This represents local regions as tokens while preserving their image positions. Two pre-normalized residual blocks contain four-head self-attention and an FFN. Each head has

$d_h = 16$:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \qquad A = \text{softmax}\Big(QK^\mathsf{T}/\sqrt{16}\Big)\,V.$$

Per image, attention weights have shape $[4, 49, 49]$. Each token attends to all 49 tokens and receives a 16-value result per head; concatenation and an output projection restore width 64. The mechanism follows scaled dot-product attention,[3] with the project's residual structure

$$\begin{aligned} U_\ell &= H_{\ell-1} + \text{Dropout}\big(\text{MHSA}(\text{LN}(H_{\ell-1}))\big), \\ H_\ell &= U_\ell + \text{FFN}(\text{LN}(U_\ell)). \end{aligned}$$

The FFN is $64 \to 256 \to 64$, with GELU and dropout 0.1. After two blocks, final layer normalization and mean pooling over 49 tokens give a 64-value vector; a $64 \to 10$ head returns logits. There is no class token.

Parameter counts are 1,088 for patch projection, 3,136 for positions, two blocks of 49,984, 128 for final normalization and 650 for the head: 104,970 in total. The project implements patchification, Q/K/V projections, attention, residual blocks and pooling with PyTorch tensor operations and layers. It does not substitute a built-in Transformer encoder or pretrained vision model.

2.4 Loss Function and Evaluation Metrics

Every model uses unweighted multiclass cross-entropy:

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(z_{i,y_i})}{\sum_{k=0}^{9}\exp(z_{i,k})}.$$

Loss is aggregated by sample count, correctly weighting the incomplete final batch. Argmax logits give labels. For a confusion matrix whose rows are true classes and columns predictions,

$$\text{Accuracy} = \frac{\sum_c C_{cc}}{\sum_{c,k} C_{ck}}, \qquad P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c},$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{MacroF1} = \frac{1}{10}\sum_{c=0}^{9} F1_c.$$

Undefined class precision, recall or F1 is assigned zero, matching `zero_division=0`. Macro-F1 weights classes equally and can reveal weaknesses hidden by overall accuracy. Tables use the mean and sample SD across three training seeds, with denominator $n - 1$. Scores converted to percent are labeled explicitly; differences between percent scores are percentage points.

2.5 Training, Hyperparameter Selection and Checkpointing

All 15 main runs use Adam, learning rate 10^{-3}, weight decay 0, batch size 64 and a 100-epoch cap. The bundle contains one effective configuration per model repeated across three seeds; it does not document an exhaustive hyperparameter search. Momentum fields present in Linear/MLP configs do not change Adam into SGD.

Linear and MLP use cosine annealing over 100 epochs to a minimum learning rate 10^{-5}, with early-stopping patience 15 and macro-F1 minimum improvement 0.001. CNN, GRU and Transformer have no scheduler and early stopping disabled. Equal epoch caps therefore do not imply equal completed budgets or training policies.

The selected checkpoint maximizes validation macro-F1, breaking exact ties by lower validation loss and earlier epoch. Test results never enter this choice. The early-stopping counter is separate: for Linear/MLP it resets only after an improvement larger than 0.001. A smaller improvement may update `best.pt` without resetting patience.

The trainer saves configs, source snapshots, metadata, environment records, history, selected metrics and checkpoints. Evaluation uses `best.pt`. The last checkpoint retains optimizer, scheduler and RNG state for supported epoch-boundary resumption; no mid-minibatch resume guarantee is claimed.

2.6 Experimental Design and Fairness Controls

This is a shared-data comparison, not matched-parameter or matched-compute experimentation. No row/column, LSTM/GRU, dropout or augmentation ablation is claimed. Such follow-ups could test proposed explanations; their outcomes are not established by this comparison.

2.7 Environment, Timing and Reproduction Protocol

Training used Google Colab Linux and an NVIDIA Tesla T4 (14.56 GiB recorded memory), PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130, NumPy 2.1.3 and CUDA 13.0. Python is 3.13.15 for 13 runs and 3.13.16 for Transformer seeds 67/69. Inputs and models use float32 without AMP. The saved settings are matmul precision `highest`, CUDA matmul

Table 2.3: Questions, controls and interpretation limits.

Focus	Changed factors	Fixed controls	Evidence/criterion
Quality	Model configuration and representation	Split, preprocessing, labels, evaluator	Accuracy, macro-F1 and CE across three seeds
Representation	Flat pixels, spatial maps, rows, patches	Image task and held-out test	Class errors; association, not isolated causality
Efficiency	Architecture, completed epochs, kernels	Batch 64 within each timing protocol	Parameters, GPU loop time, separate CPU forward time
Seed variation	Run seed	Per-model effective config	Mean/sample SD and paired seed differences

TF32 disabled, cuDNN TF32 allowed and cuDNN benchmarking disabled; deterministic algorithms are enabled. Python patch-version variation is a reproducibility limitation, not a verified cause of score differences.

Validation replay and final-test evaluation used a separate Colab CPU session: Python 3.13.16, PyTorch 2.11.0+cpu, torchvision 0.26.0+cpu, NumPy 2.1.3 and two threads. Replay accuracy/F1 match the original validation scores in all runs; the largest loss difference is 3.84×10^{-8}.

Reported inference speed uses the separately retained **Windows CPU profile**, measured on 24 October 2026: Python 3.14.6, PyTorch 2.13.0+cpu, torchvision 0.28.0+cpu, NumPy 2.5.2 and two threads. The recorded processor is Intel64 Family 6 Model 170 Stepping 4, GenuineIntel (AMD64); no friendly product name was recorded. These are not Colab or T4 speed measurements.

For each selected checkpoint, the forward-only protocol uses a preallocated zero tensor $[64, 1, 28, 28]$, float32, evaluation mode, 20 warm-ups and 100 timed forwards. It excludes checkpoint I/O, loading and transfer. Per-run latency is the median batch time; amortized latency divides that median by 64; throughput is 64 divided by the median in seconds. Tables average these run-level values and give sample SD across three seeds. This is not batch-one or end-to-end speed.

3 Implementation Results

3.1 Completed Runs and Evidence Coverage

The evidence contains five models at seeds 69420, 67 and 69: 15 main runs and 30 best/last checkpoints. All runs record clean source revision `a8539be`, protocol `a1-v0`, the same seed-36 split and the same normalization identity. Eleven runs reached 100 epochs. Linear completed 38/33/31 epochs and MLP seed 67 completed 83, consistent with their configured stopping policy. Full IDs and per-seed results are in Appendix B.

The training notebook originally produced validation-only results. A separate postprocessing stage restored every selected checkpoint, replayed validation and then evaluated the official test partition. Its 30 evaluations each contain 10,000 predictions, totaling 300,000 saved rows. These are repeated evaluations of the same validation/test images, not 300,000 independent samples.

Artifact checks verified 61 completion markers, source/config/checkpoint identities, prediction IDs, class support, confusion matrices, per-class scores and seed aggregates. Accuracy/F1 replay matches training validation exactly; the largest loss change is 3.84×10^{-8}. Lack of GPU availability in that evaluation session caused its GPU timing stage to be skipped, not its test evaluation. The separately retained Windows CPU timing profile supplies the inference measurements reported below. No additional training was needed for these stages.

3.2 Main Quantitative Comparison

Table 3.1 reports selected-checkpoint validation and official-test metrics, using all three seeds without selecting a test winner. Validation supports model selection and is not an independent final assessment.

Table 3.1: Mean ± sample SD over three seeds. Accuracy/F1 are percentages; CE is sample-weighted cross-entropy.

Model	Split	Accuracy (%)	Macro-F1 (%)	CE loss
Linear	Val.	79.267 ± 0.176	79.254 ± 0.159	0.58673 ± 0.00234
Linear	Test	78.377 ± 0.283	78.427 ± 0.307	0.61142 ± 0.00292
MLP	Val.	90.333 ± 0.165	90.288 ± 0.157	0.26488 ± 0.00057
MLP	Test	89.720 ± 0.104	89.698 ± 0.094	0.28806 ± 0.00258
CNN	Val.	91.547 ± 0.115	91.556 ± 0.056	0.24148 ± 0.00427
CNN	Test	90.690 ± 0.324	90.726 ± 0.262	0.26421 ± 0.00855
GRU	Val.	91.063 ± 0.061	91.055 ± 0.064	0.26549 ± 0.00438
GRU	Test	90.297 ± 0.255	90.281 ± 0.250	0.28357 ± 0.00235
Transformer	Val.	91.580 ± 0.026	91.548 ± 0.011	0.23699 ± 0.00250
Transformer	Test	91.267 ± 0.237	91.234 ± 0.215	0.24440 ± 0.00402

The Transformer has the highest mean test accuracy (91.267%) and macro-F1 (91.234%), compared with 90.690% and 90.726% for CNN. The differences are 0.577 and 0.508 percentage points. Paired Transformer–CNN test-accuracy differences are $+0.81, +0.62, +0.30$ points at seeds 69420, 67 and 69. This is consistent across the three observed seeds, but does not establish statistical significance or superiority on other datasets.

The MLP improves substantially over Linear (89.720% versus 78.377% test accuracy), and GRU reaches 90.297%. Across these runs, test-accuracy sample SD ranges from 0.104 to 0.324 percentage points. The small number of seeds limits inference about the wider run distribution. Figure 3.1 visualizes the same test means and SDs.

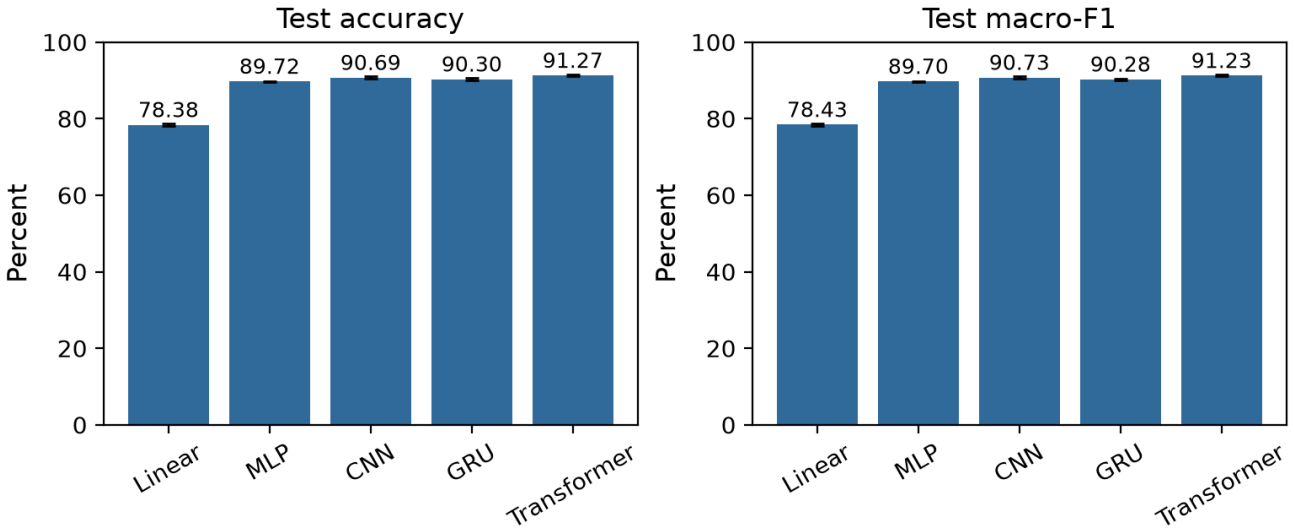


Figure 3.1: Official-test accuracy and macro-F1: mean with sample-SD error bars, $n = 3$. Both axes retain the 0–100 percent scale.

Table 3.2 separates total, trainable and frozen parameter elements. All five recorded architectures are fully trainable. Buffers are not parameters. CNN uses about 72.9% fewer parameters than Transformer while remaining close in predictive quality; the larger MLP does not obtain the strongest scores.

Table 3.2: Exact parameter counts of the reported configurations.

Model	Total	Trainable	Frozen
Linear	7,850	7,850	0
MLP	235,146	235,146	0
CNN	28,426	28,426	0
GRU	61,962	61,962	0
Transformer	104,970	104,970	0

3.3 Learning Dynamics and Convergence

Figures 3.2–3.6 show seed 67 for each model. The vertical line marks the saved best-validation checkpoint, using 1-based epochs. The selected epoch need not be the final epoch.

Linear stabilizes at much lower F1 and fluctuates after early improvement. MLP, CNN and Transformer improve rapidly before making smaller late gains; their observed training and validation curves remain relatively close. These training curves include augmentation and

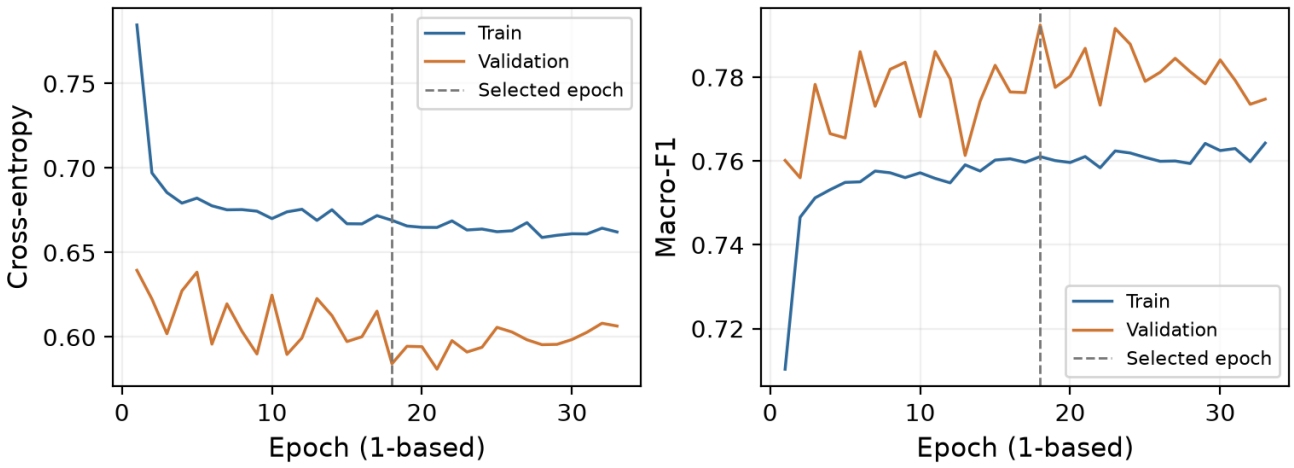


Figure 3.2: Linear learning history, seed 67: training/validation loss and macro-F1.

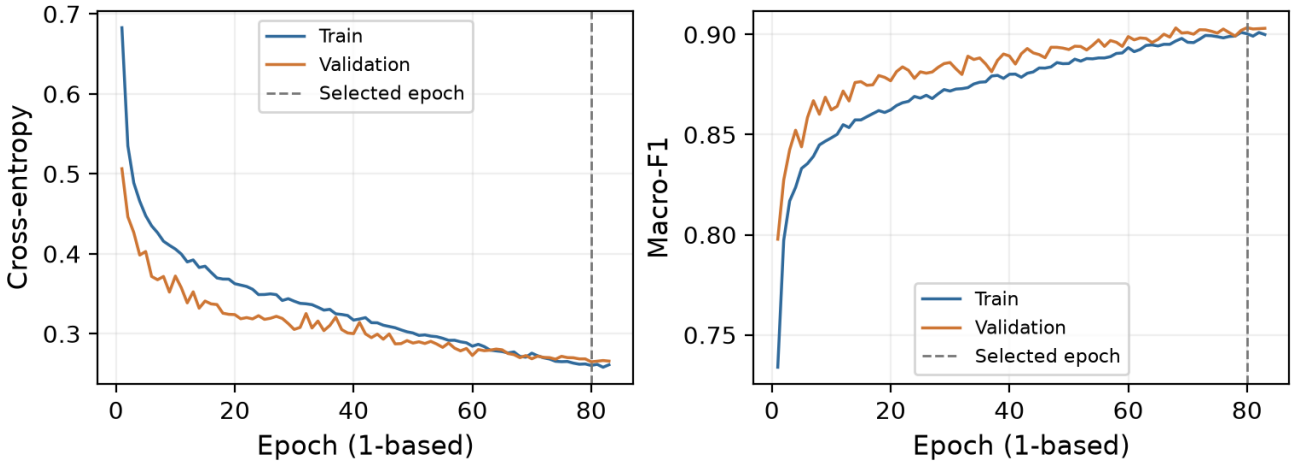


Figure 3.3: MLP learning history, seed 67: training/validation loss and macro-F1.

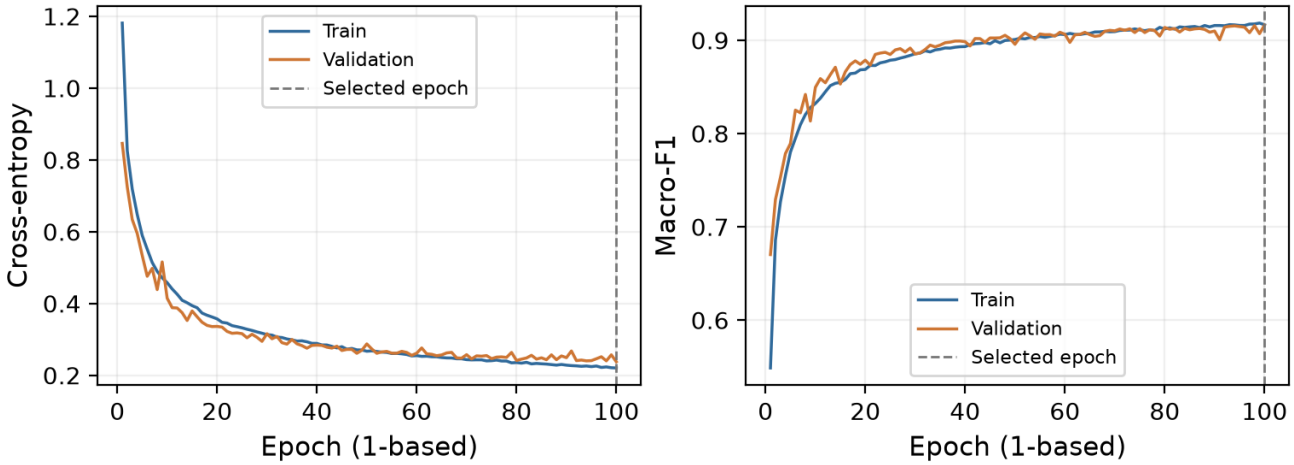


Figure 3.4: CNN learning history, seed 67: training/validation loss and macro-F1.

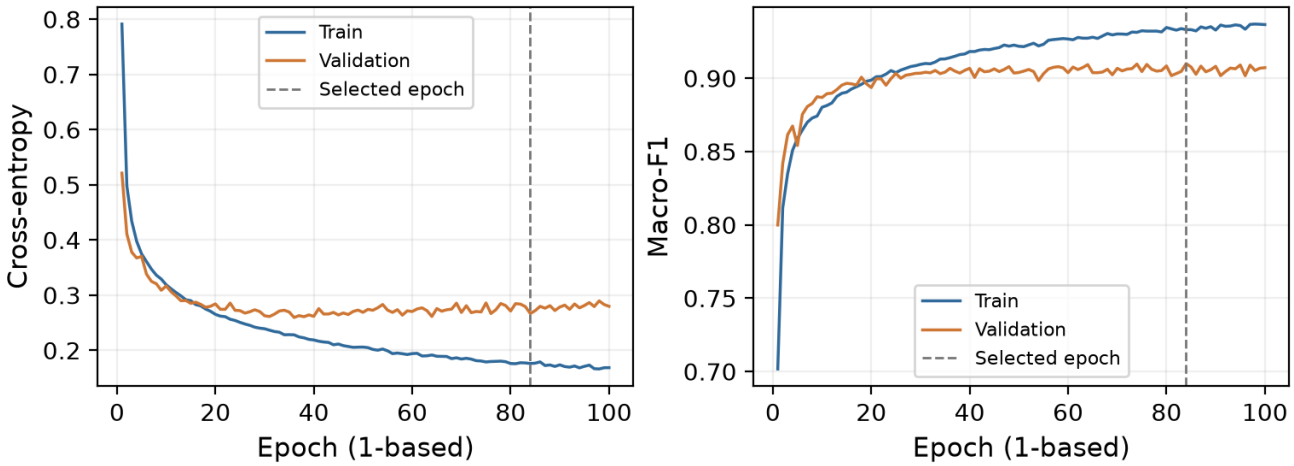


Figure 3.5: GRU learning history, seed 67: training/validation loss and macro-F1.

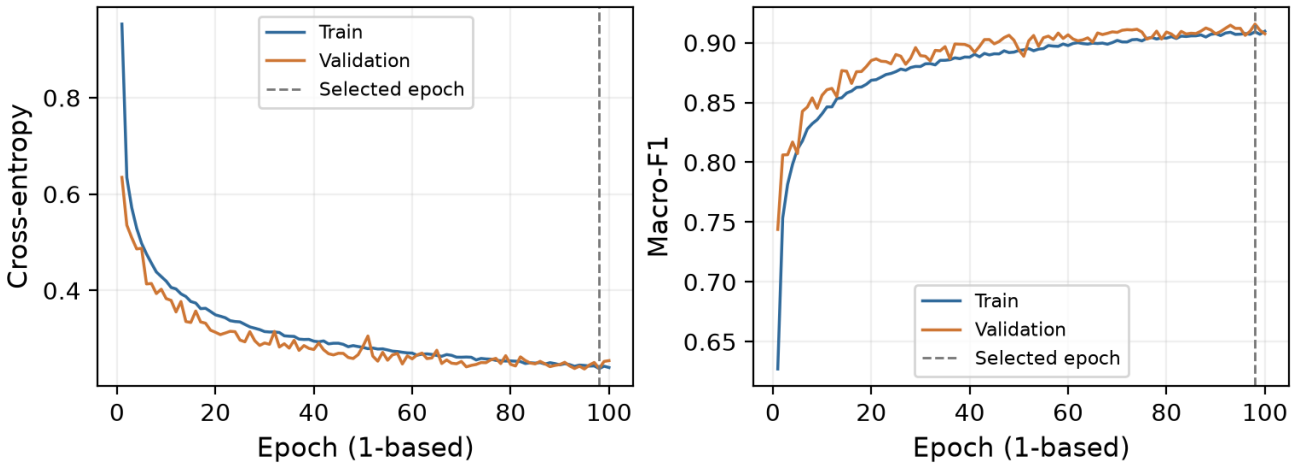


Figure 3.6: Transformer learning history, seed 67: training/validation loss and macro-F1.

dropout effects, while validation uses deterministic preprocessing and evaluation mode. GRU shows the clearest late divergence: training F1 continues improving while validation F1 largely plateaus and validation loss rises slightly. Its seed-67 selected epoch is 84, before the final epoch 100. This pattern suggests that continuing optimization does not keep improving held-out validation performance. It does not prove a single causal mechanism. Best epochs for the other seed-67 runs are Linear 18, MLP 80, CNN 100 and Transformer 98.

3.4 Efficiency and Accuracy–Compute Trade-offs

Table 3.3 sums the saved `epoch_seconds` values, each of which surrounds training plus validation. It excludes full startup, preparation and checkpoint overhead, and should not be read as isolated forward/backward time.

Table 3.3: T4 train-and-validation epoch-loop cost. Epoch sequences and best epochs are ordered by seeds 69420/67/69; times are mean ± sample SD.

Model	Epochs	Loop time (min)	Best epochs
Linear	38 / 33 / 31	10.27 ± 1.05	30 / 18 / 16
MLP	100 / 83 / 100	29.85 ± 3.18	92 / 80 / 93
CNN	100 / 100 / 100	33.49 ± 0.44	100 / 100 / 81
GRU	100 / 100 / 100	32.89 ± 0.24	62 / 84 / 53
Transformer	100 / 100 / 100	38.39 ± 0.91	96 / 98 / 100

The Transformer has the largest observed loop cost (38.39 minutes), compared with CNN's 33.49 minutes, a difference of about 4.90 minutes. Both use 100 epochs, but their per-epoch computation differs. Linear/MLP totals also reflect their early-stopping policy and, for MLP, a shorter 83-epoch run. The MLP's timing SD therefore includes a budget difference; it is not solely machine noise or variation in model quality.

Table 3.4 reports the separate Windows CPU measurements under Section 2.7's batch-64 forward-only protocol. A row averages three per-checkpoint medians of 100 timed batches; SD describes those three run-level values.

Table 3.4: Windows CPU forward-only timing: two threads, float32, batch 64, 20 warm-ups and 100 timed batches per checkpoint; $n = 3$.

Model	Batch time (ms)	ms/image	Images/s
Linear	0.097 ± 0.003	0.001523 ± 0.000051	657237 ± 22106
MLP	1.151 ± 0.129	0.017987 ± 0.002016	56047 ± 6034
CNN	25.875 ± 2.663	0.404296 ± 0.041606	2491 ± 260
GRU	6.897 ± 2.059	0.107765 ± 0.032176	9776 ± 2494
Transformer	28.463 ± 3.584	0.444738 ± 0.056000	2273 ± 288

The Linear and MLP have much lower measured CPU cost than the other models. CNN averages 0.404296 ms/image versus Transformer's 0.444738, so the Transformer is about 10.0% slower by these observed means. Their timing SDs are substantial, and no formal claim of a stable speed ordering is made. GRU's mean is 0.107765 ms/image, giving a different quality–speed trade-off from its T4 training cost. Throughput averages per-run rates; it need not equal the reciprocal of the rounded aggregate latency.

These figures describe the documented CPU/library profile and preallocated input, not end-to-end service performance. Parameter count alone does not predict runtime: kernel structure, sequence processing and implementation matter, but their individual contributions were not isolated. Deployment choices require measurements on the intended hardware, batch size and input path.

3.5 Confusion Matrices and Class-Level Results

The following figures use the seed-67 official-test predictions; class metrics quoted in the accompanying text refer to those same predictions. They are representative single-run evidence; Table 3.1 remains the three-seed comparison, and Appendix C reports aggregate class metrics. Every figure uses the same raw-count color range, 0–1,000, and the same class order.

There are 1,000 test examples per class. Consequently, $\text{Recall}_c = C_{cc}/1000$; raw diagonal counts themselves are not recalls. Because supports are equal, overall accuracy equals mean class recall. Quoted precision, recall and F1 use the 0–1 scale. Full seed-level class metrics are retained in the source CSV, and aggregate class tables appear in the appendix.

3.5.1 Linear

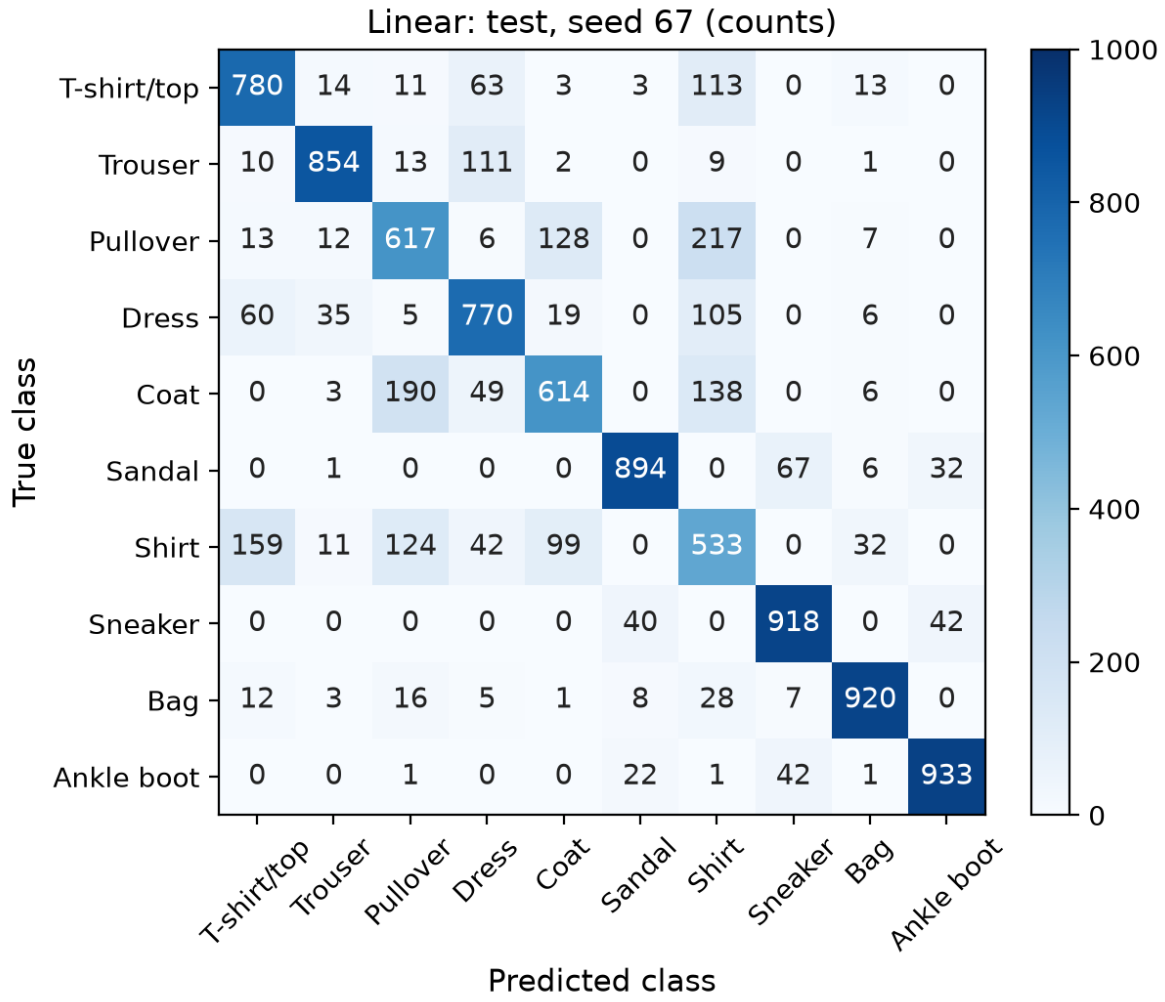


Figure 3.7: Linear test confusion counts, seed 67; shared color scale 0–1,000.

Linear's weakest class is Shirt (F1 0.497), with 159 Shirts predicted as T-shirt/top, 124 as Pullover and 99 as Coat. Pullover/Coat confusion is also large: 128 Pullovers are predicted as Coat and 190 Coats as Pullover. This motivates using nonlinear and spatial feature models, but the comparison does not isolate representation from all other configuration differences.

3.5.2 MLP

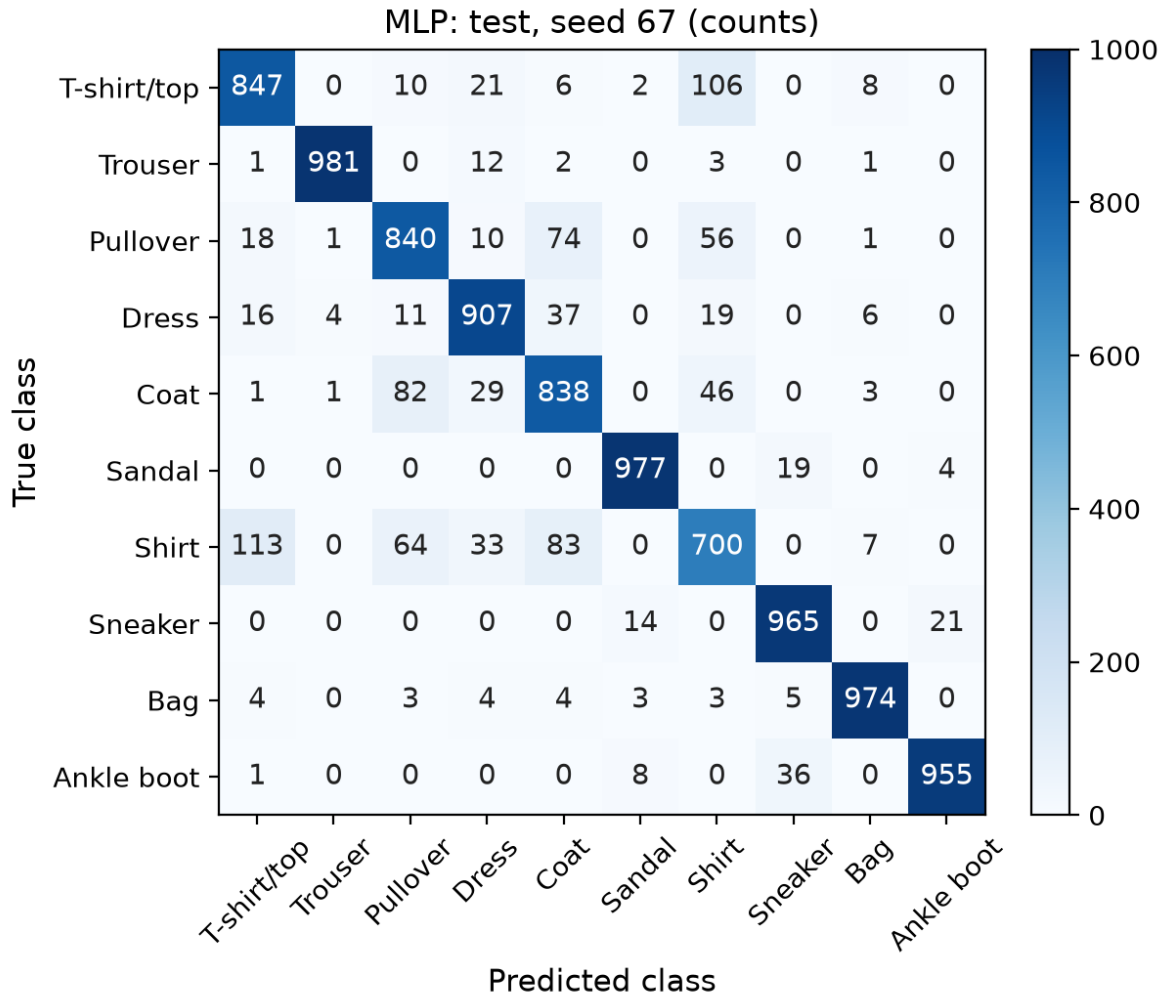


Figure 3.8: MLP test confusion counts, seed 67; shared color scale 0–1,000.

MLP improves overall accuracy to 89.84% in this run. Trouser has F1 0.987, while Shirt remains lowest at 0.724. There are 113 Shirt-to-T-shirt/top errors and 106 in the reverse direction. Pullover–Coat errors persist (74 and 82 in the two directions), showing that greater dense capacity does not eliminate apparel ambiguity.

3.5.3 CNN

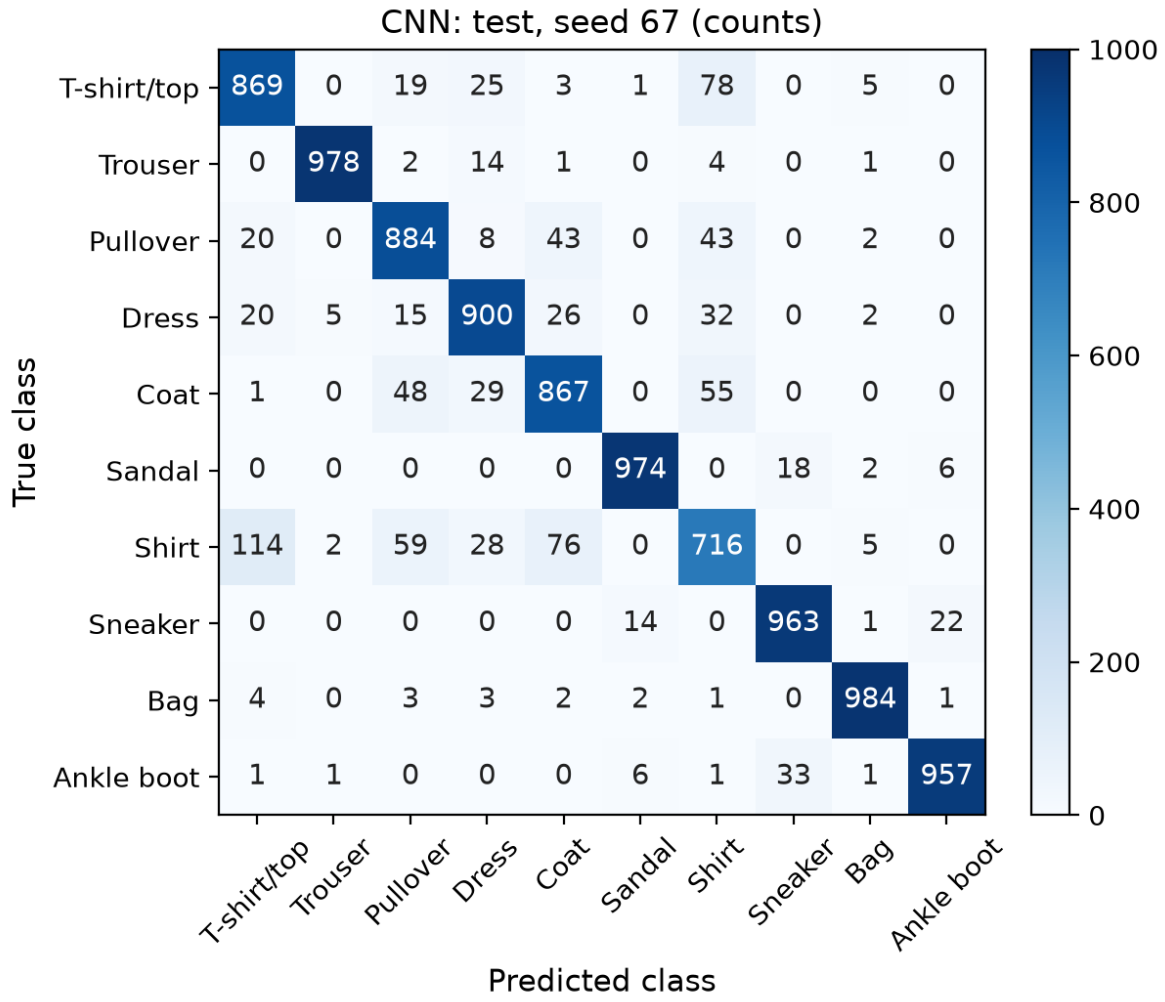


Figure 3.9: CNN test confusion counts, seed 67; shared color scale 0–1,000.

CNN reaches 90.92% accuracy in this seed. Shirt remains weakest (F1 0.742); 114 Shirts are predicted as T-shirt/top, 76 as Coat and 59 as Pullover. Pullover–Coat errors are lower than MLP's in both directions (43 and 48). This is compatible with useful local image features, but is not a controlled causal test of locality.

3.5.4 GRU

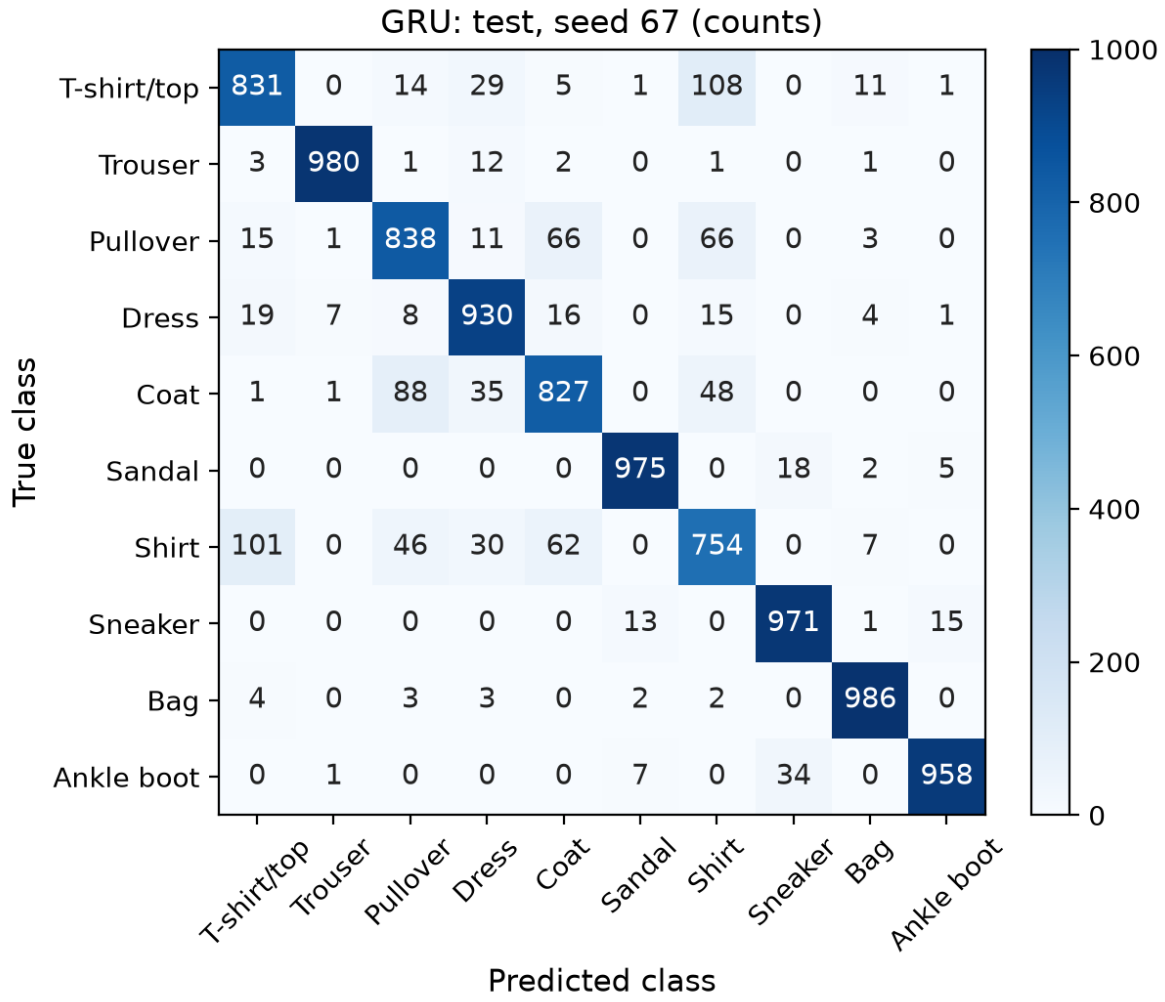


Figure 3.10: GRU test confusion counts, seed 67; shared color scale 0–1,000.

GRU reaches 90.50% accuracy. Bag has its highest recall, 0.986; Dress recall is 0.930. GRU has the highest Shirt recall among these five seed-67 models (0.754), although Shirt is still its lowest-F1 class (0.756). Its overall score therefore does not reveal every class-level advantage.

3.5.5 Transformer

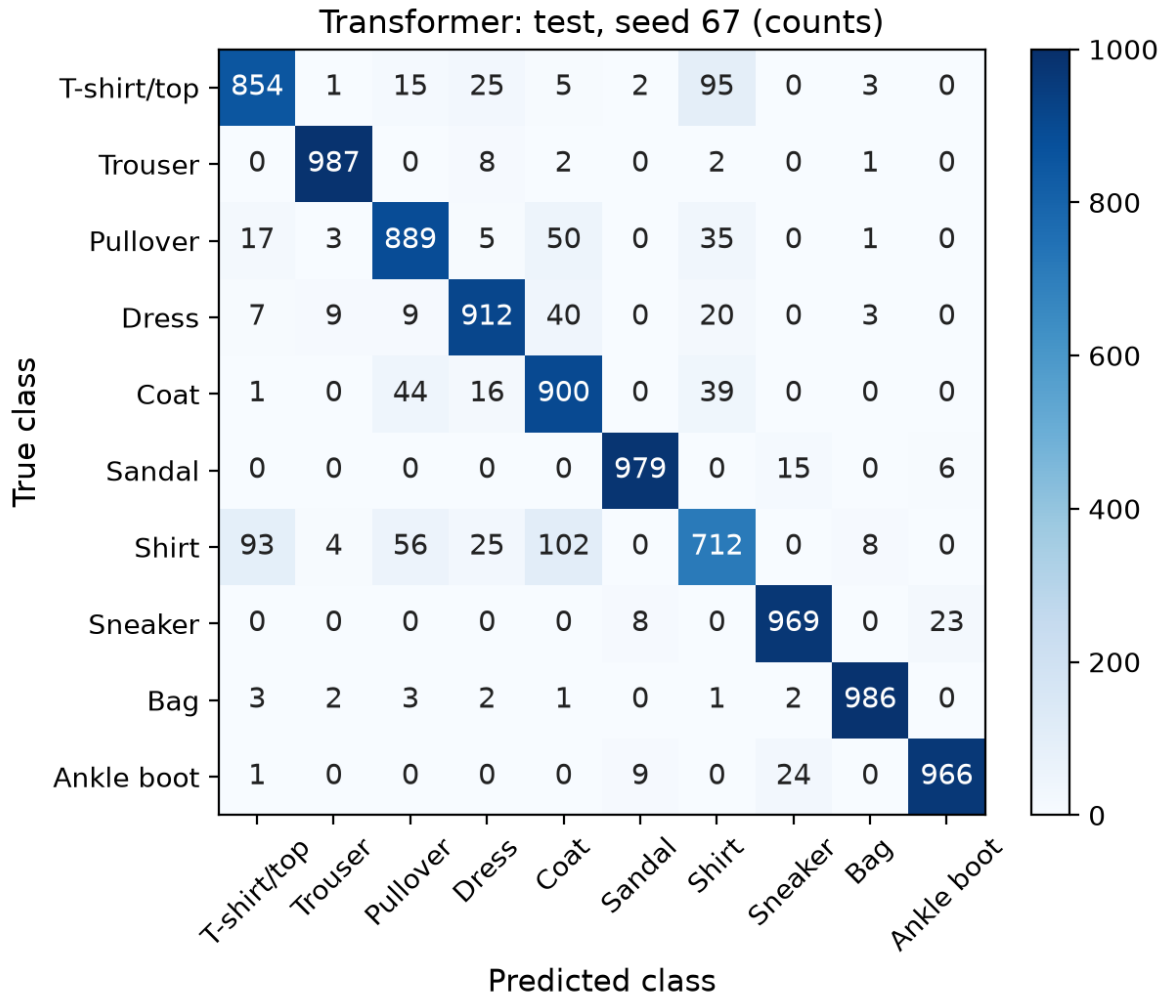


Figure 3.11: Transformer test confusion counts, seed 67; shared color scale 0–1,000.

Transformer reaches 91.54% accuracy in seed 67 and has strong Bag and Trouser F1 (0.985 and 0.984). Shirt remains weakest (F1 0.748), with 102 Shirts predicted as Coat, 93 as T-shirt/top and 56 as Pullover. Shirt precision is higher than GRU's while recall is lower (0.712), illustrating a class-level trade-off despite the strongest aggregate score.

3.6 Qualitative Predictions and Error Analysis

Each gallery below contains three correct, three difficult-correct and three confidently incorrect examples from the same seed-67 test run. Correct examples use the common IDs 0, 1 and 10 for all models; difficult-correct examples are the three lowest-confidence correct predictions per model, and errors are the three highest-confidence mistakes. Other IDs therefore vary across models. The IDs identify official-test images; probabilities are rounded to three decimals. These selected galleries illustrate behavior, not the prevalence of an error type.

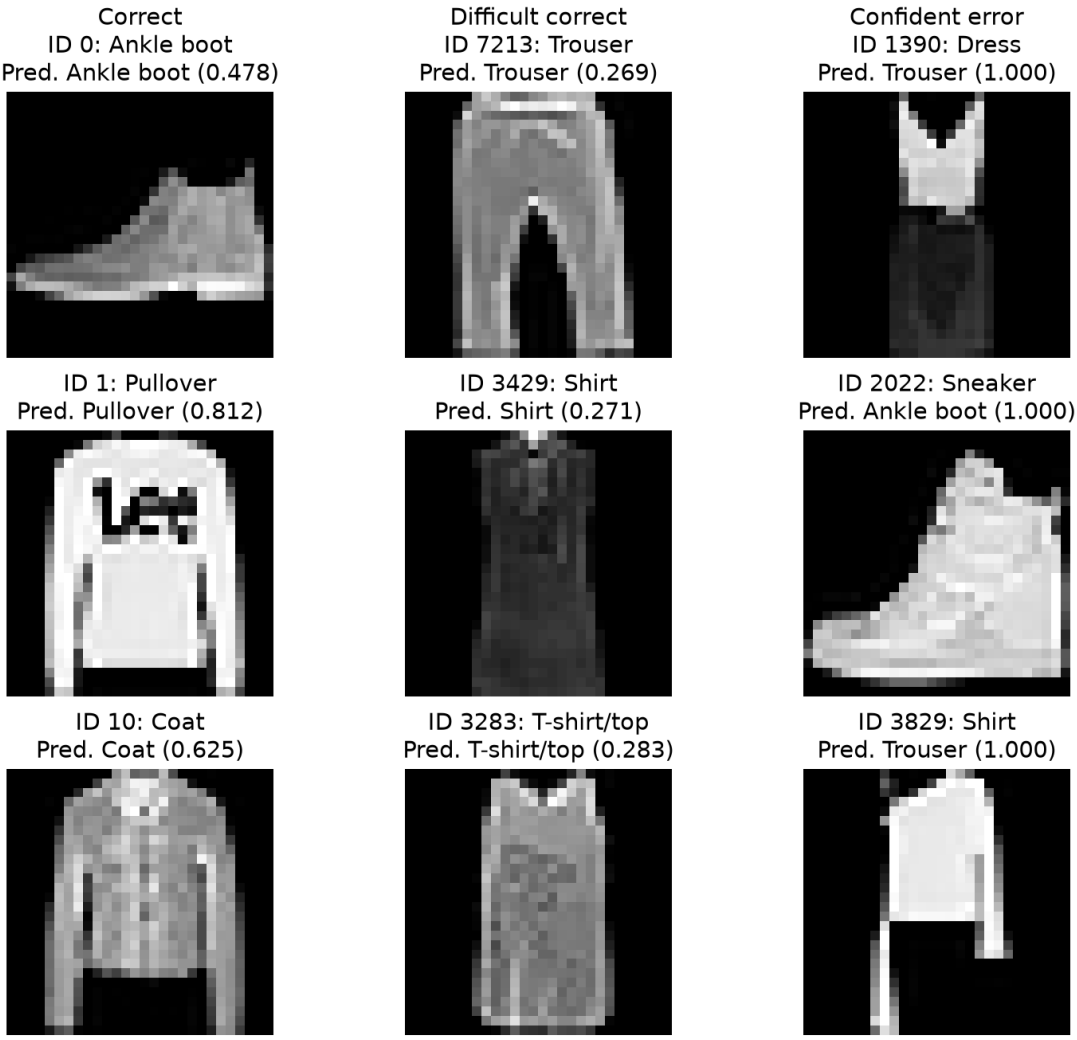


Figure 3.12: Linear: common correct examples (left), low-confidence correct cases (middle) and high-confidence errors (right), test seed 67.



Figure 3.13: MLP: common correct examples (left), low-confidence correct cases (middle) and high-confidence errors (right), test seed 67.

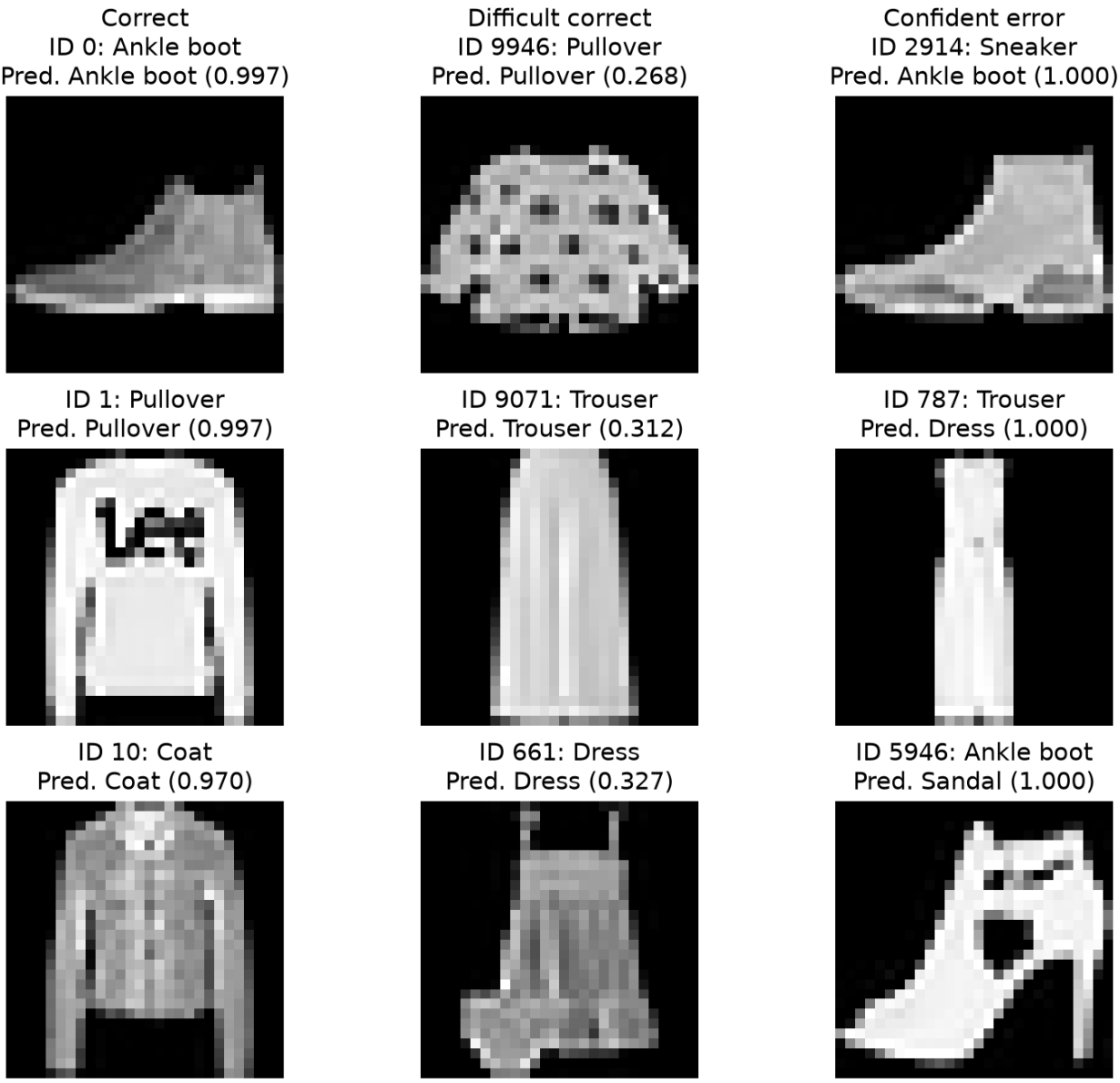


Figure 3.14: CNN: common correct examples (left), low-confidence correct cases (middle) and high-confidence errors (right), test seed 67.

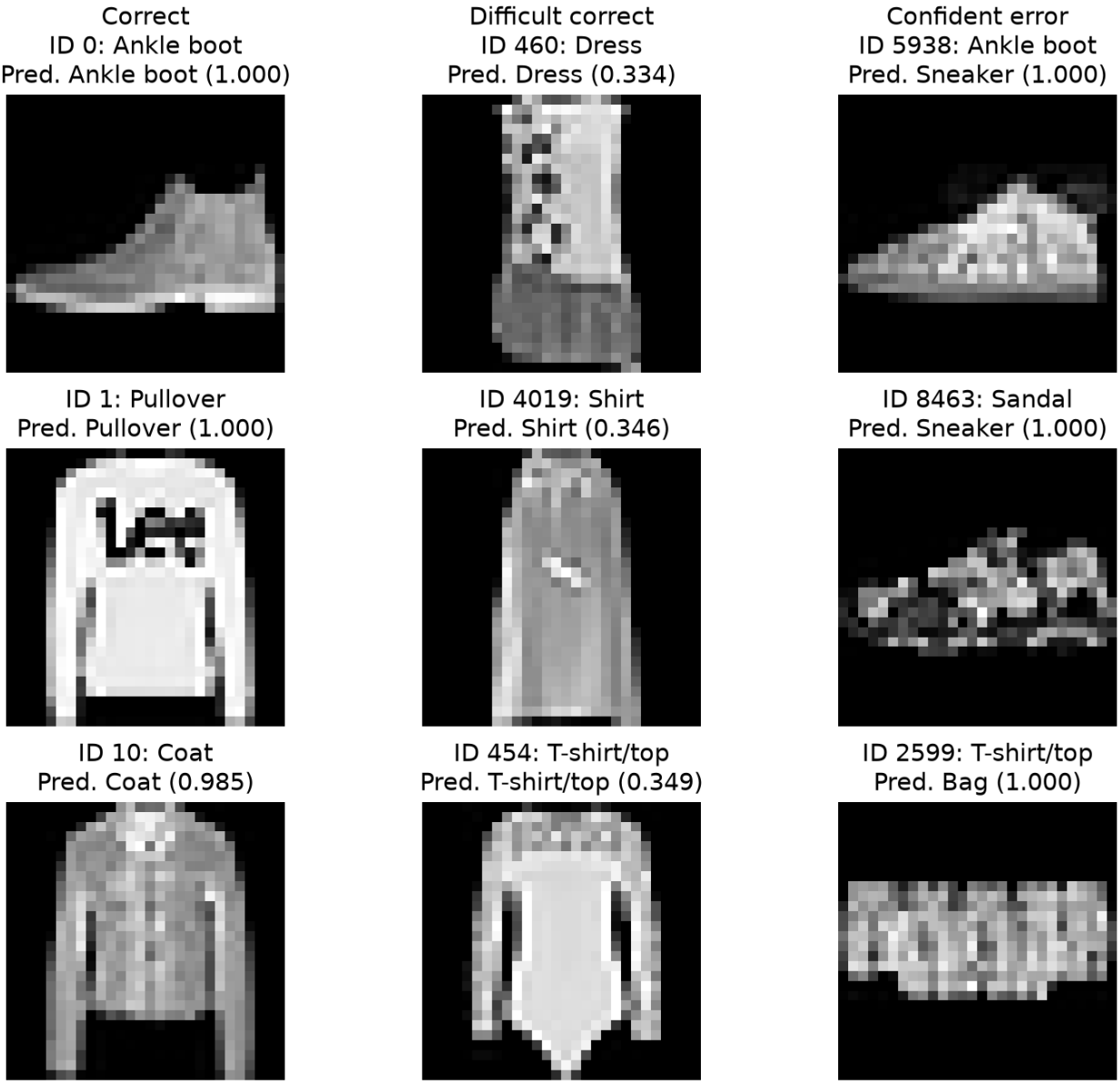


Figure 3.15: GRU: common correct examples (left), low-confidence correct cases (middle) and high-confidence errors (right), test seed 67.

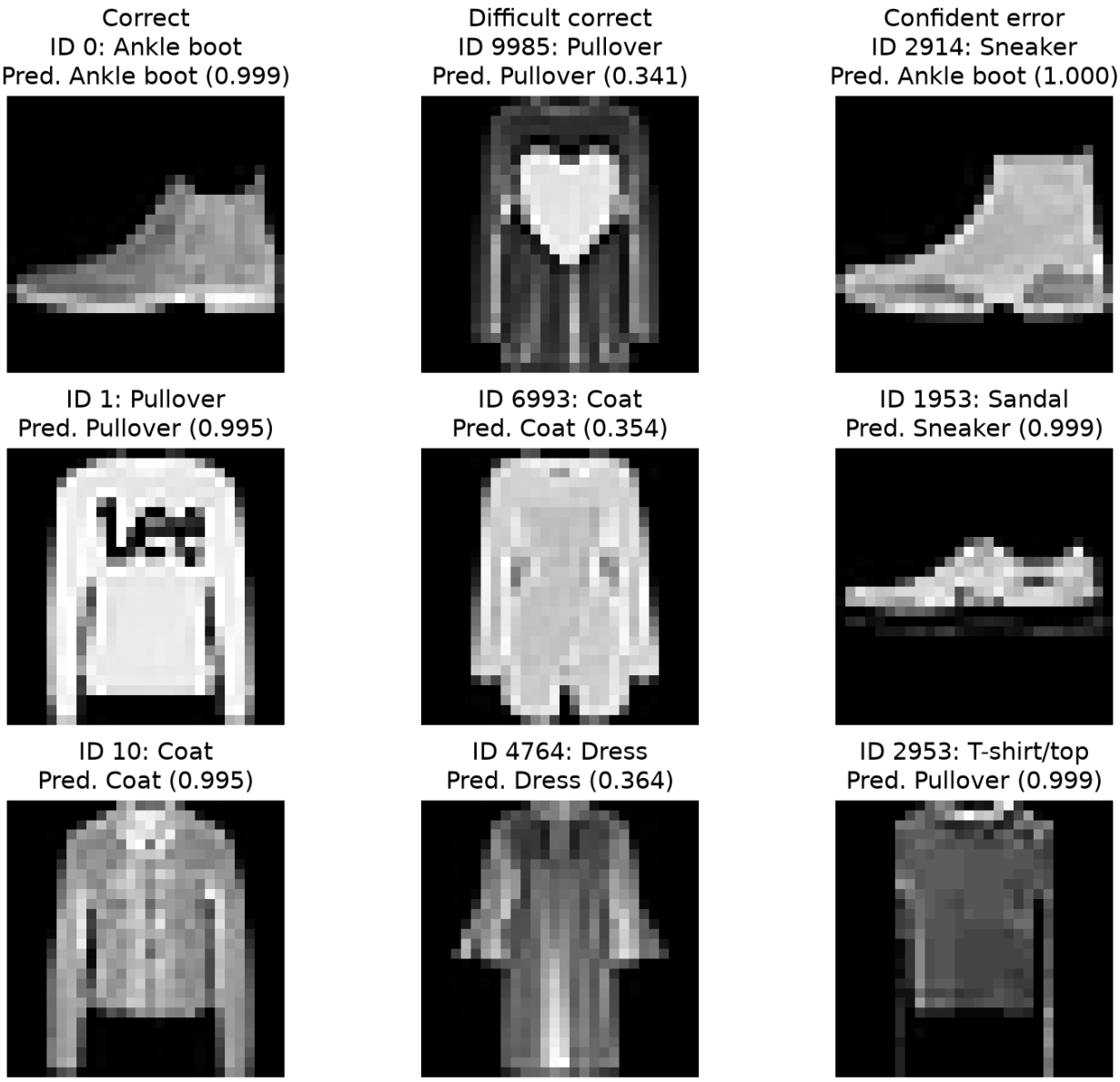


Figure 3.16: Transformer: common correct examples (left), low-confidence correct cases (middle) and high-confidence errors (right), test seed 67.

The common example ID 0 (Ankle boot) is correct with confidence approximately 0.478 for Linear and near 1.0 for the nonlinear models. Conversely, both MLP and CNN confidently predict Dress for true Trouser ID 787. Thus, a high softmax score does not guarantee correctness. Low-confidence correct cases also show that uncertainty and the final argmax label are distinct.

The class matrices quantify recurrent upper-body confusions; footwear galleries show additional Sandal/Sneaker/Ankle-boot ambiguity. Plausible explanations are shared silhou-

ettes, limited resolution and missing texture cues, but the present artifacts do not prove label errors or identify a unique cause. Several focused follow-ups would address these patterns:

- Apparel confusions: inspect validated examples of Shirt/T-shirt/top/Coat/ Pullover and compare representations under matched training/parameter budgets. This would test whether local or sequence features explain the observed gap.
- Footwear confusions: measure pair-specific error rates and test a controlled augmentation change while keeping model initialization and evaluation fixed. The present galleries alone cannot establish whether augmentation would help.
- Confident mistakes: assess reliability or calibration error on validation, then evaluate a validation-fitted procedure under a fresh or preregistered held-out protocol. A gallery is insufficient to claim population calibration.

These are proposed investigations, not completed ablations or promised gains.

3.7 Representation and Inductive-Bias Discussion

Linear assigns each class one global weighted pixel combination, with no hidden feature hierarchy. Its low scores and severe apparel confusions are consistent with limited expressiveness. Adding nonlinear dense layers improves test accuracy by about 11.34 percentage points, but MLP remains below CNN and Transformer despite having the largest parameter count.

The CNN's local connectivity and shared filters suit spatial image patterns. It achieves close-to-leading quality with only 28,426 parameters. GRU instead aggregates an ordered row sequence; its high seed-67 Shirt recall shows that some class-level strengths are not captured by the overall ranking. No row/ column comparison was run, so the contribution of traversal order is unisolated.

The Transformer can combine information from distant patches through attention, using learned position parameters to distinguish their locations. Its best aggregate quality does not remove the shared Shirt/Coat ambiguity. The results support these configurations as practical alternatives, but do not prove that attention, locality or recurrence alone caused the measured differences.

3.8 Limitations

The evidence covers Fashion-MNIST, one effective configuration per family, three seeds and one shared split. There is no matched-budget ablation, external dataset, robustness

benchmark or significance test. The repeated evaluations reuse the same images, and selecting checkpoints on validation makes validation scores optimistic for final comparison. Epoch-loop cost combines training/validation and differs in completed epochs. Training, classification evaluation and inference timing use distinct environments. The Windows timing profile uses preallocated zeros and excludes I/O/transfer; its amortized batch-64 cost does not predict batch-one or real-service latency. Python patch-version variation, library differences and unrecorded friendly CPU model name further limit exact cross-machine reproduction. The raw Windows timing samples are retained; no unverified Colab timing table is substituted for them.
Example selection is deterministic but selective; images are not an uncertainty or calibration sample. No perceptual-duplicate removal or independent label audit was performed. The proposed explanations and follow-ups remain hypotheses.

3.9 Conclusion

The four research questions can be answered within these limits:

1. **Predictive quality.** Transformer leads at 91.267% test accuracy and 91.234% macro-F1; CNN follows at 90.690% and 90.726%.

2. **Representation and errors.** Spatial and patch-token configurations perform strongly, but similar apparel remains difficult. Architecture alone was not isolated as the cause.

3. **Efficiency.** CNN has 72.9% fewer parameters and 4.90 fewer minutes of T4 epoch-loop time than Transformer. Their Windows CPU times are 0.404/0.445 ms per image; the preferred trade-off depends on the device.

4. **Seed consistency.** Test-accuracy SD is at most 0.324 percentage points. All three paired Transformer–CNN differences are positive, without establishing statistical significance.

The measured outcome is a configuration-specific trade-off on Fashion-MNIST, not a general ordering of architecture families for image classification.

Reproducibility and Artifact Access

The report follows the handbook's three-part structure and distinguishes measured results from proposed follow-ups.[8] The source repository is Deep-Learning-Benchmark-Suite; the assignment entry point is the A1 project page. The reported run revision is:

`a8539bee778d99e2902a1dd36aa157c6ebb3a42c`

The run artifacts and snapshots, rather than an older README status paragraph, define the effective configurations. Appendix B maps all 15 results to their run IDs; the source package includes exact effective JSON configs, per-run metrics, raw timing samples and input/checkpoint hashes under `evidence/`.

Original checkpoints are maintained in the group's Drive artifact bundle `MyDrive/dlbench_a1_main/main/`, with one run directory per ID. They are not embedded in the LaTeX archive and no anonymous public checkpoint download URL is asserted. Readers without that bundle can reconstruct the models by training the included effective configs at the pinned source revision. This provides a reconstruction route rather than claiming current access permissions to the group's Drive.

For a Linux/Colab CUDA environment, the following installation identifies the main recorded tensor-library builds. Python patch versions and operational differences remain as described in Section 2.7.

```
git clone https://github.com/TuanKhai1210/Deep-Learning-Benchmark-Suite.git
cd Deep-Learning-Benchmark-Suite
git checkout a8539bee778d99e2902a1dd36aa157c6ebb3a42c
python -m pip install torch==2.11.0 torchvision==0.26.0 --index-url https://download.pytorch.
    org/whl/cu130
python -m pip install numpy==2.1.3
python -m pip install -e ".[ml]"
```

Official previous-version installation instructions document these builds.[6] The effective configs and the environment records govern reconstruction; installing unpinned optional extras alone is not a complete environment lock.

The included `reproduction/reproduce.py` reads a selected saved config, requires the pinned repository revision and changes only explicit operational paths/device settings. Its default action prints the resolved configuration; training requires the explicit action below. In this example the report source archive has been extracted to `/content/A1_Final`:

```
python /content/A1_Final/reproduction/reproduce.py \
  --repo-root . --model cnn --seed 67 --action train \
  --data-root data --output-root runs/a1_reproduction --device cuda
```

Repeat with the five model names `linear, mlp, cnn, rnn, transformer` and seeds 69420, 67, 69. A new run may differ numerically because exact cross-machine or cross-version bitwise reproduction has not been established. Evaluation of a trusted saved checkpoint requires an explicit action and path:

```
python /content/A1_Final/reproduction/reproduce.py \
  --repo-root . --model cnn --seed 67 --action test \
  --checkpoint /content/drive/MyDrive/dlbench_a1_main/main/a1_cnn_main_seed67_20261023-030133
      _702c4e/best.pt \
  --data-root data --device cpu
```

The source package also retains the one-pass evaluation notebook and timing-only notebook in `reproduction/`. Their outputs preserve split/device labels and completion identities. Regenerating report tables and figures from existing artifacts uses `tools/build_assets.py`; it performs no model training or checkpoint inference.

Member Contributions and AI Usage Disclosure

The task division follows the project's recorded ownership: Nguyen Hao Thien is responsible for data/EDA/preprocessing and Linear/CNN; Nguyen Anh Khoa for the shared training/evaluation engine, MLP/GRU and metrics/checkpointing; Ta Tuan Khai for Transformer, configuration/artifacts, integration and report reproducibility. Shared interfaces and results require cross-member review. The cover sheet retains the group's supplied completion declarations; automated artifact checks do not independently measure individual contribution percentages.

AI assistance was used and is disclosed below. Historical details come from the repository's `AI_USAGE.md`, copied into the source package as `evidence/AI_USAGE_source.md`. Model/version labels in that log are reported as recorded rather than independently certified service identifiers.

The historical log names Codex 5.6 Sol and GPT-6 in selected entries, Gemini 3.8 Flash and 3.1 Pro (High), and ChatGPT 5.6 Luna. Other serving model IDs were not recorded. Representative prompt summaries include explaining leakage-free EDA and stratified splits (Thien), identifying states needed for training resumption and generating metric/trainer tests (Khoa), and making artifacts traceable to configs/checkpoints and completing the LaTeX report from verified runs (Khai). Historical log entries record student checks against the handbook, source interfaces, PyTorch documentation and tests. They do not establish that AI output is correct without those checks.

For the 23–26 October report/analysis work, Khai used Codex; the exact serving model

Table 3.5: Recorded AI tools, users and development scope.

Tool/model as recorded	Member	Stage and affected work
OpenAI Codex	Ta Tuan Khai	Aug.–Oct. 2026: protocol, artifacts, Transformer support, analysis, source correction and report generation
Gemini	Nguyen Hao Thien; Nguyen Anh Khoa	Sep. 2026: data/EDA, metrics/checkpoints, trainer tests and analysis notebook
GitHub Copilot	Nguyen Anh Khoa	Sep. 2026: MLP/trainer tests and optimizer support
OpenAI ChatGPT	Nguyen Anh Khoa	Sep. 2026: trainer test-generation support

identifier for every interaction was not retained. AI proposed and edited report prose, generated aggregation/plotting helpers, corrected source/report inconsistencies, and helped assemble the final document. The current automated verification recomputes aggregates from saved predictions/metrics, checks source and checkpoint identities, verifies raw dataset resources, and compiles/inspects the PDF. Official documentation and papers support the theoretical descriptions. The team remains responsible for the interpretation and final verification of its submission; this disclosure does not assert an unrecorded human sign-off. Additional session detail is retained in `evidence/AI_USAGE_report.md`.

References

[1] H. Xiao, K. Rasul and R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. https://arxiv.org/abs/1708.07747.

[2] Zalando Research. *Fashion-MNIST repository and MIT License*. 2017. https://github.com/zalandoresearch/fashion-mnist/blob/master/LICENSE. Accessed 26 October 2026.

[3] A. Vaswani et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/abs/1706.03762.

[4] PyTorch contributors. *torch.nn.GRU, PyTorch 2.11 documentation*. https://docs.pytorch.org/docs/2.11/generated/torch.nn.GRU.html. Accessed 26 October 2026.

[5] PyTorch contributors. *CrossEntropyLoss, PyTorch 2.11 documentation*. https://docs.pytorch.org/docs/2.11/generated/torch.nn.CrossEntropyLoss.html. Accessed 26 October 2026.

[6] PyTorch contributors. *Installing previous versions of PyTorch*. https://pytorch.org/get-started/previous-versions/. Accessed 26 October 2026.

[7] Tiki-taka. *Deep Learning Benchmark Suite*, source revision a8539be. https://github.com/TuanKhai1210/Deep-Learning-Benchmark-Suite.

[8] CO3133, Semester 261. *Course Project Handbook*, revision 14 September 2026. Course-provided document; Sections 3–5 and 8–14.

A Full Configurations and Architecture Details

The source package contains all 15 effective config snapshots under `evidence/configs/`. Within each model, seeds change but architecture, data/evaluation settings and training policy remain fixed. The common protocol uses Fashion-MNIST, split seed 36, batch 64, Adam at 10^{-3}, weight decay 0, a 100-epoch cap and validation macro-F1 checkpoint selection.

Table A.1: Effective model/training settings; detailed snapshots are included in the source.

Model	Architecture	Model-specific training/regularization
Linear	784 to 10	Cosine to 10^{-5}; patience 15, min-delta 0.001
MLP	784–256–128–10; ReLU	Hidden dropout 0.1; same cosine/patience as Linear
CNN	Channels 32/64; 3-by-3 kernels; max pooling; 64–128–10 head	Head dropout 0.1; no scheduler/early stop
GRU	Rows; input 28; hidden 128; one layer, one direction	No inter-layer dropout; no scheduler/early stop
Transformer	Patch 4; width 64; four heads; two blocks; FFN width 256	Dropout 0.1; no scheduler/early stop

The saved configs also identify operational paths and the source configuration modules. Device/path overrides used for later CPU stages are recorded separately, not silently presented as the original T4 training setup. Parameter elements are counted once through the model's parameter iterator; buffers are excluded and `requires_grad` determines trainability.

B Per-Seed Results and Run Provenance

Table B.1 reports all seeds. Accuracy/F1 are percentages and the selected epoch is 1-based; the source history itself uses 0-based indices. There is no best-test-seed selection. The following exact run IDs link these rows to their histories, effective configs and best checkpoints. Their timestamp and random suffix are artifact identifiers, not additional experimental factors. The source commit and common identities recorded by the bundle are:

Git revision `a8539bee778d99e2902a1dd36aa157c6ebb3a42c`

Split SHA-256 `e4a776305b2a1b5c392ba196d023de004f47124e8f831495b9733c48e72c25e6`

Statistics SHA-256 `e47b863cb866cdc61613bb88d772d02f8aedc0a21fa56b79dee27b27d341381e`

Table B.1: All 15 selected checkpoints and their evaluation results.

Model	Seed	Epochs	Best	Val. acc.	Test acc.	Test F1	Test CE
Linear	69420	38	30	79.16	78.12	78.098	0.61253
Linear	67	33	18	79.17	78.33	78.478	0.60811
Linear	69	31	16	79.47	78.68	78.706	0.61362
MLP	69420	100	92	90.15	89.67	89.647	0.29002
MLP	67	83	80	90.38	89.84	89.806	0.28514
MLP	69	100	93	90.47	89.65	89.641	0.28903
CNN	69420	100	100	91.43	90.32	90.423	0.27405
CNN	67	100	100	91.66	90.92	90.885	0.25862
CNN	69	100	81	91.55	90.83	90.870	0.25997
GRU	69420	100	62	91.13	90.38	90.366	0.28147
GRU	67	100	84	91.01	90.50	90.478	0.28611
GRU	69	100	53	91.05	90.01	90.000	0.28312
Transformer	69420	100	96	91.57	91.13	91.120	0.24628
Transformer	67	100	98	91.61	91.54	91.482	0.23978
Transformer	69	100	100	91.56	91.13	91.100	0.24713

Model	Seed	Run ID
Linear	69420	`a1_linear_main_seed69420_20261023-002758_9761a8`
Linear	67	`a1_linear_main_seed67_20261023-003924_612a81`
Linear	69	`a1_linear_main_seed69_20261023-004922_8fcd80`
MLP	69420	`a1_mlp_main_seed69420_20261023-005847_49b2dc`
MLP	67	`a1_mlp_main_seed67_20261023-013028_2a1c4c`
MLP	69	`a1_mlp_main_seed69_20261023-015641_53b249`
CNN	69420	`a1_cnn_main_seed69420_20261023-022827_cd3858`
CNN	67	`a1_cnn_main_seed67_20261023-030133_702c4e`
CNN	69	`a1_cnn_main_seed69_20261023-064233_c73732`
GRU	69420	`a1_rnn_main_seed69420_20261023-071609_51281c`
GRU	67	`a1_rnn_main_seed67_20261023-074850_b7dcdb`
GRU	69	`a1_rnn_main_seed69_20261023-082158_329a4e`
Transformer	69420	`a1_transformer_main_seed69420_20261023-085501_3eb003`
Transformer	67	`a1_transformer_main_seed67_20261024-030332_1727fa`
Transformer	69	`a1_transformer_main_seed69_20261024-034117_45d824`

Table B.2: Exact run identifiers. The `rnn` artifact name denotes the GRU configuration.

Full per-run checkpoint/config hashes are retained in `evidence/report_facts.json` and the original run metadata.

C Additional Class-Level Results

The tables below aggregate official-test precision, recall and F1 over the three seeds as mean ± sample SD, expressed in percent. Support is 1,000 examples per class per run. The same test images are evaluated repeatedly; the support is not 3,000 independent images. Validation and per-seed class results are also included in the machine-readable `evidence/per_class.csv`.

Table C.1: Linear official-test per-class metrics, mean ± sample SD, $n = 3$.

Class	Precision (%)	Recall (%)	F1 (%)	Support
T-shirt/top	77.12 ± 1.72	76.90 ± 1.35	76.99 ± 0.26	1000
Trouser	89.83 ± 3.06	88.57 ± 2.74	89.15 ± 1.54	1000
Pullover	60.56 ± 3.01	66.00 ± 4.35	63.07 ± 2.20	1000
Dress	75.08 ± 1.77	79.03 ± 2.55	76.99 ± 1.51	1000
Coat	71.49 ± 1.08	56.17 ± 5.76	62.80 ± 3.74	1000
Sandal	92.84 ± 1.18	88.73 ± 1.42	90.73 ± 0.21	1000
Shirt	47.41 ± 0.79	51.80 ± 1.30	49.50 ± 0.28	1000
Sneaker	86.42 ± 2.05	93.53 ± 1.53	89.81 ± 0.42	1000
Bag	93.28 ± 0.54	91.50 ± 0.44	92.38 ± 0.11	1000
Ankle boot	94.28 ± 1.69	91.53 ± 1.98	92.86 ± 0.23	1000

Table C.2: MLP official-test per-class metrics, mean ± sample SD, $n = 3$.

Class	Precision (%)	Recall (%)	F1 (%)	Support
T-shirt/top	84.73 ± 0.22	84.53 ± 0.47	84.63 ± 0.32	1000
Trouser	99.22 ± 0.38	97.67 ± 0.38	98.44 ± 0.33	1000
Pullover	84.06 ± 0.98	82.67 ± 1.40	83.35 ± 0.24	1000
Dress	88.89 ± 0.33	90.67 ± 0.15	89.77 ± 0.19	1000
Coat	80.11 ± 0.26	84.20 ± 0.40	82.11 ± 0.23	1000
Sandal	97.62 ± 0.29	96.93 ± 0.75	97.27 ± 0.31	1000
Shirt	73.95 ± 0.95	70.53 ± 1.01	72.19 ± 0.43	1000
Sneaker	94.18 ± 0.06	96.63 ± 0.12	95.39 ± 0.08	1000
Bag	97.30 ± 0.10	97.43 ± 0.06	97.37 ± 0.03	1000
Ankle boot	97.00 ± 0.49	95.93 ± 0.38	96.46 ± 0.17	1000

Table C.3: CNN official-test per-class metrics, mean ± sample SD, $n = 3$.

Class	Precision (%)	Recall (%)	F1 (%)	Support
T-shirt/top	86.32 ± 2.79	84.27 ± 3.42	85.21 ± 0.47	1000
Trouser	99.26 ± 0.21	97.57 ± 0.59	98.40 ± 0.20	1000
Pullover	87.28 ± 1.26	87.20 ± 1.04	87.23 ± 0.16	1000
Dress	89.51 ± 1.05	89.20 ± 1.06	89.35 ± 0.31	1000
Coat	85.34 ± 0.25	85.13 ± 1.40	85.23 ± 0.61	1000
Sandal	98.18 ± 0.44	96.63 ± 1.59	97.39 ± 0.69	1000
Shirt	73.15 ± 3.82	76.13 ± 4.20	74.47 ± 0.42	1000
Sneaker	94.17 ± 2.03	96.57 ± 1.52	95.33 ± 0.36	1000
Bag	98.36 ± 0.24	98.20 ± 0.35	98.28 ± 0.11	1000
Ankle boot	96.75 ± 0.88	96.00 ± 1.08	96.37 ± 0.11	1000

Table C.4: GRU official-test per-class metrics, mean ± sample SD, $n = 3$.

Class	Precision (%)	Recall (%)	F1 (%)	Support
T-shirt/top	85.39 ± 0.87	82.33 ± 1.50	83.82 ± 0.43	1000
Trouser	99.06 ± 0.40	98.13 ± 0.32	98.59 ± 0.10	1000
Pullover	84.19 ± 0.98	83.73 ± 0.21	83.96 ± 0.58	1000
Dress	88.91 ± 0.41	91.33 ± 1.47	90.10 ± 0.62	1000
Coat	83.43 ± 1.10	84.60 ± 1.85	83.99 ± 0.36	1000
Sandal	97.76 ± 0.31	97.27 ± 0.40	97.51 ± 0.34	1000
Shirt	74.46 ± 1.34	73.97 ± 1.29	74.21 ± 1.31	1000
Sneaker	94.77 ± 0.81	97.23 ± 0.32	95.99 ± 0.56	1000
Bag	97.21 ± 0.77	98.67 ± 0.12	97.93 ± 0.34	1000
Ankle boot	97.72 ± 0.07	95.70 ± 0.66	96.70 ± 0.37	1000

Table C.5: Transformer official-test per-class metrics, mean ± sample SD, $n = 3$.

Class	Precision (%)	Recall (%)	F1 (%)	Support
T-shirt/top	87.10 ± 0.55	85.50 ± 0.17	86.29 ± 0.18	1000
Trouser	98.73 ± 0.54	98.50 ± 0.53	98.62 ± 0.33	1000
Pullover	86.37 ± 1.04	88.27 ± 0.93	87.31 ± 0.79	1000
Dress	91.04 ± 2.87	91.27 ± 2.00	91.11 ± 0.54	1000
Coat	83.06 ± 1.48	87.10 ± 2.95	84.99 ± 0.67	1000
Sandal	98.20 ± 0.17	98.00 ± 0.17	98.10 ± 0.17	1000
Shirt	77.20 ± 1.44	72.67 ± 1.37	74.85 ± 0.49	1000
Sneaker	95.58 ± 0.58	96.47 ± 0.84	96.02 ± 0.37	1000
Bag	98.34 ± 0.06	98.47 ± 0.15	98.40 ± 0.10	1000
Ankle boot	96.89 ± 0.69	96.43 ± 0.38	96.66 ± 0.22	1000

D Detailed Verification and Reproduction Notes

The saved-output audit of 24 October 2026 verified 382 files and 61 completed stages in the original CPU evaluation profile, independently recomputing metrics from 300,000 prediction rows across 30 evaluations. Class order, argmax labels, support, confusion counts, gallery selection and sample IDs agree. Mean negative log true-class probability matches the saved cross-entropy within 10^{-6}. The replay agreement supports using the CPU evaluations of the T4-selected checkpoints; it is not evidence of bitwise equivalence on every platform.

The retained Windows timing profile, `cpu-f7208f3550fc`, contains all 15 checkpoint identities and 100 raw batch timings per run. Its session and per-run records are copied into the source package. The original CPU evaluation profile, `cpu-0bd6ba8e08e9`, explicitly skipped GPU timing. A separate Colab timing summary appeared in an intermediate report without its raw session records; this final document uses the retained Windows measurements instead. No new inference or training was run to construct the final tables. Report-generation checks on 26 October compare all classification aggregates with the retained evaluation summary, verify checkpoint hashes, inspect official IDX/gzip identities and class counts, regenerate figures from saved outputs, and check LaTeX references and layout. The example preprocessing batch is newly regenerated data, clearly separated from original training history.

Source line endings deserve care: the committed split JSON and the Windows checkout can have different byte hashes because of LF/CRLF conversion while parsing to the same indices. Provenance comparisons use the recorded committed blob identity and semantic membership, rather than silently relabeling an unrelated checkout-file hash.

The editable content is organized under `sections/`, `tables/` and `figures/`. The `evidence/` directory retains effective configs, per-run/class aggregates, raw Windows timing samples, resource hashes and figure selections. The `reproduction/` directory provides the pinned-source reconstruction helper and standalone Colab evaluation/timing notebooks. Together, these records distinguish original measurements from later report-generation checks.